

フィルタバンク理論に基づく畳込み辞書学習

～ 構造制約を利用した畳込みネットワーク構築 ～

村松正吾（新潟大学工学部工学科電子情報通信プログラム）

電子情報通学会 基礎・境界ソサイエティ Fundamentals Review

https://www.jstage.jst.go.jp/article/essfr/17/2/17_116/_article/-char/ja

動作環境：MATLAB R2025a 以降（テキスト形式 Live Script、R2026a で動作確認）

準備

```
clear
close all

nsoltDic = ""; % "nsoltdictionary_20230621232700260"

isCodegen = false; % コード生成
setup(isCodegen)
```

```
SaivDr-4.2.2.5 exits.
Skip code generation
```

パラメータ設定

- ブロックサイズ
- 冗長度
- スパース度

```
% Block size
szBlk = [ 8 8 ];

% Redundancy ratio for RICA/K-SVD
redundancyRatio = 7/3;

% Sparsity ratio
sparsityRatio = 3/64;
```

画像の読込

- $\mathbf{y} \in \mathbb{R}^N$

```
% 原画像の準備
file_yorg = "../data/yorg.png";
if ~exist(file_yorg, 'file')
    unzip('http://www.ess.ic.kanagawa-it.ac.jp/std_img/monoimage2/Mono-Image2.zip', '../results')
    yfull = imread('../results/Mono-Image2/512X512/barbara512.bmp');
    ycrop = yfull(1:192,end-255:end);
    imwrite(ycrop, file_yorg)
end

% 原画像の読み込み
yorg = im2double(imread(file_yorg));
szOrg = size(yorg);
```

画像表示

```
figure
imshow(yorg);
title('Original image y')
```



零平均化

```
%ymean = mean(y,"all");
%y = yorg - ymean;
meansubtract = @(x) x-mean(x,"all");
y = meansubtract(yorg);
```

離散コサイン変換 (DCT)

$$[\mathbf{C}_M]_{k,n} = \sqrt{\frac{2}{M}} \alpha_k \cos \frac{k(n+1/2)\pi}{M}, \quad k, n = 0, 1, \dots, M-1$$

$$\alpha_k = \begin{cases} \frac{1}{\sqrt{2}} & k = 0 \\ 1 & k = 1, 2, \dots, M-1 \end{cases}$$

基底画像

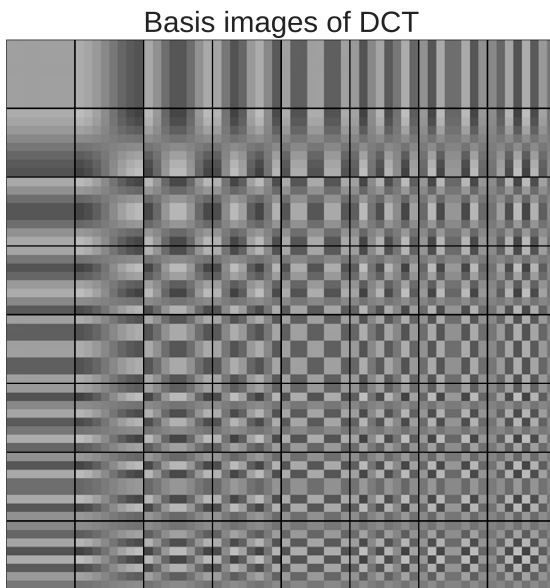
$$\mathbf{B}_{k,\ell} = \mathbf{C}_M^{-1} \mathbf{E}_{k,\ell} \mathbf{C}_M^{-T}, \quad k, \ell = 0, 1, \dots, M-1$$

$$\mathbf{E}_{k,\ell} = \mathbf{e}_k \mathbf{e}_\ell^T$$

```
basisImagesDct = zeros(szBlk(1),szBlk(2),prod(szBlk));
iBasis = 1;
for iRow=1:szBlk(1)
    for iCol=1:szBlk(2)
        E = zeros(szBlk);
        E(iRow,iCol) = 1;
        basisImagesDct(:, :, iBasis) = idct2(E, szBlk(1), szBlk(2));
        iBasis = iBasis + 1;
    end
end
```

基底画像の表示

```
figure
montage(imresize(basisImagesDct,8,'nearest')+0.5,'BorderSize',[2 2])
title('Basis images of DCT')
```



ブロック **DCT** による合成処理とその随伴処理の定義

```
syn_blkdst = @(x) blockproc(x,szBlk,@(block_struct)
idct2(block_struct.data));
adj_blkdst = @(y) blockproc(y,szBlk,@(block_struct) dct2(block_struct.data));
```

随伴関係の確認

```
x = adj_blkdst(y);
v = randn(size(x));
u = syn_blkdst(v);
assert(abs(dot(y(:),u(:))-dot(x(:),v(:)))<1e-9)
```

主成分分析 (PCA)

問題設定:

直交性と次元削減

$$\Phi^T \Phi = \mathbf{I}_M, \forall b, \forall p, \|\mathbf{x}_b\|_0 \leq p < M$$

を制約条件とした最小自乗問題

$$\{\hat{\Phi}, \{\hat{\mathbf{x}}_b\}_b\} = \arg \min_{\{\Phi, \{\mathbf{x}_b\}_b\}} \frac{1}{2S} \sum_{b=1}^S \|\mathbf{y}_b - \Phi \mathbf{x}_b\|_2^2$$

を解く. 上式は等価的に

$$\hat{\Phi} = \arg \max_{\Phi} \text{tr}(\Phi_{:,0:p-1}^T \hat{\Sigma}_y \Phi_{:,0:p-1}) \quad \text{s.t.} \quad \Phi^T \Phi = \mathbf{I}_M$$

と表現できる. ただし, $\hat{\Sigma}_y$ は観測ベクトル $\{\mathbf{y}_b\}_b$ (零平均を仮定) の標本分散共分散行列である.

解:

固有値分解

$$\hat{\Phi}^T \hat{\Sigma}_y \hat{\Phi} = \Lambda$$

ただし, $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_M)$. $\lambda_1 \geq \lambda_2 \geq \dots \lambda_M$ は $\hat{\Sigma}_y$ の固有値.

画像 $\mathbf{y}$ からのデータ行列 $\mathbf{Y}$ の生成

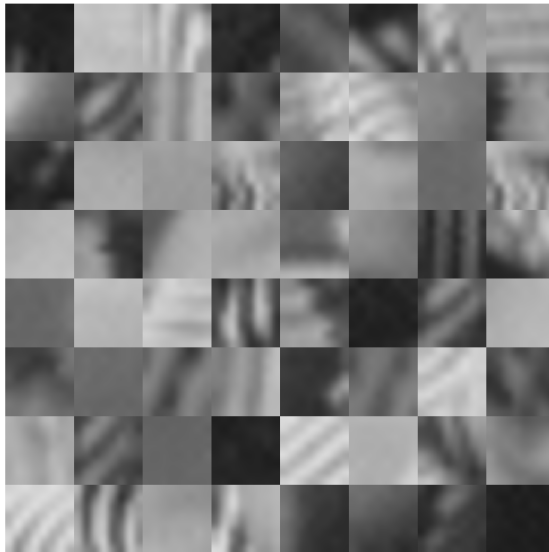
標本平均ブロックを引く代わりに, 予め零平均化したデータで学習

```
nPatches = 20*prod(szOrg./szBlk); % PCA/RICA/K-SVD 学習用のパッチをランダム抽出
npos = randsample(prod(szOrg-szBlk),nPatches);
ybs = zeros(szBlk(1),szBlk(2),nPatches,'like',y);
szSrchy = szOrg(1)-szBlk(1);
```

```

for iPatch = 1:nPatches
    ny_ = mod(npos(iPatch)-1,szSrchy)+1;
    nx_ = floor((npos(iPatch)-1)/szSrchy)+1;
    ybs(:, :, iPatch) = y(ny_:ny_+szBlk(1)-1,nx_:nx_+szBlk(2)-1);
end
figure
montage(ybs+0.5, 'Size', [8 8]);

```



```

drawnow

Y = reshape(ybs,prod(szBlk),[]);

```

標本分散共分散行列 $\hat{\Sigma}_y$ の計算

```
SigmaY = cov(Y.');
```

標本分散共分散行列 $\hat{\Sigma}_y$ の固有値分解

```
[Phi_pca,Lambda] = eig(SigmaY);
```

固有値 λ の大きさの降順に列ベクトルをソート (Sorting column vectors in the descending order of the eigenvalues λ)

```
[~,idx] = sort(diag(Lambda), 'descend');
Phi_pca = Phi_pca(:,idx);
```

固有ベクトルを基底画像に変換

```
nBases = prod(szBlk);
```

```

basisImagesPca = zeros(szBlk(1),szBlk(2),nBases);
for iBasis = 1:nBases
    basisImagesPca(:,:,iBasis) =
    reshape(Phi_pca(:, iBasis),szBlk(1),szBlk(2));
end

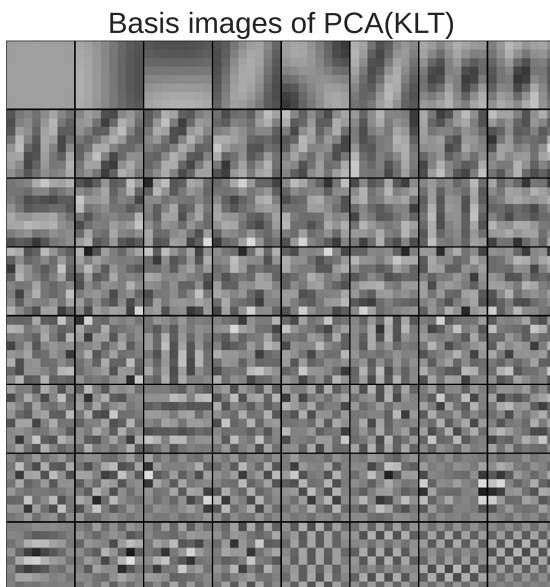
```

基底画像の表示（辞書）

```

figure
montage(imresize(basisImagesPca,8,'nearest')+.5,'BorderSize',[2 2])
title('Basis images of PCA(KLT)')

```



ブロック **PCA** による合成処理とその随伴処理の定義

```

syn_blkpca = @(x) col2im(Phi_pca*x,szBlk,szOrg,"distinct");
adj_blkpca = @(y) Phi_pca.'*im2col(y,szBlk,"distinct");

```

随伴関係の確認

```

x = adj_blkpca(y);
v = randn(size(x));
u = syn_blkpca(v);
assert(abs(dot(y(:),u(:))-dot(x(:),v(:)))<1e-9)

```

再構成独立成分分析（**RICA**）

問題設定:

$$\hat{\Phi} = \arg \min_{\Phi} \frac{1}{2S} \sum_{b=1}^S \|y_b - \Phi \Phi^T y_b\|_2^2 + \frac{\alpha}{S} \sum_{b=1}^S \rho(\Phi^T y_b)$$

$$= \arg \min_{\Phi} \frac{(2\alpha)^{-1}}{S} \sum_{b=1}^S \|\mathbf{y}_b - \Phi \Phi^T \mathbf{y}_b\|_2^2 + \frac{1}{S} \sum_{b=1}^S \rho(\Phi^T \mathbf{y}_b)$$

ただし、 $\{\mathbf{y}_n\}_n \subset \mathbb{R}^M$, $\Phi = (\phi_1, \phi_2, \dots, \phi_P) \in \mathbb{R}^{M \times P}$, $M \geq P$ である。

参考文献:

Le, Quoc V., Alexandre Karpenko, Jiquan Ngiam, and Andrew Y. Ng. "ICA with Reconstruction Cost for Efficient Overcomplete Feature Learning." Advances in Neural Information Processing Systems. Vol. 24, 2011, pp. 1017–1025. <https://papers.nips.cc/paper/4467-ica-with-reconstruction-cost-for-efficient-overcomplete-feature-learning.pdf>.

パラメータ設定

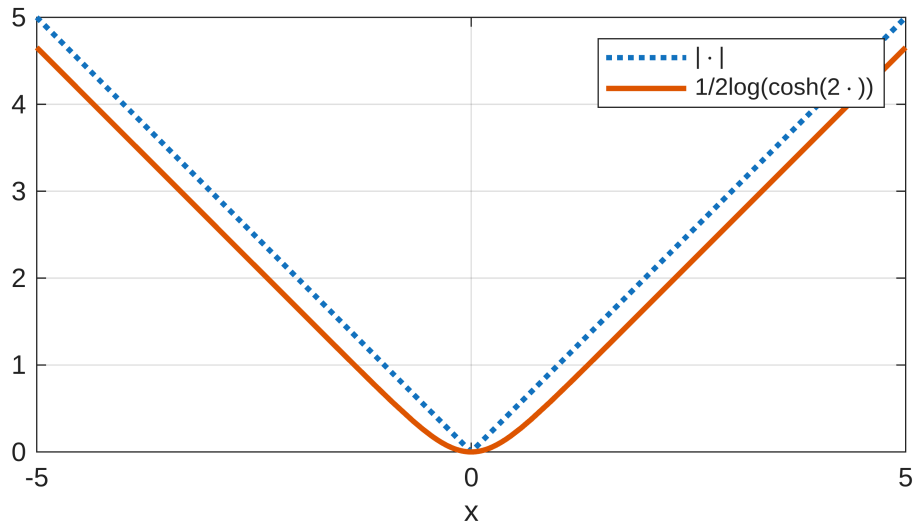
- 繰返し回数 (Number of iterations)
- 正則化パラメータ (Regularization parameter)

```
% Number of iterations
nItersRica = 1e5;
% Regularization parameter
alpha = 2e-3;
```

コントラスト関数の例

$$\rho(\Phi^T \mathbf{y}) := \frac{1}{2} \sum_{p=1}^P \log \circ \cosh(2\phi_p^T \mathbf{y})$$

```
figure
fplot(@(x) abs(x), [-5 5], ':', 'LineWidth', 2, 'DisplayName', '| \cdot |')
hold on
fplot(@(x) log(cosh(2*x))/2, [-5
5], '-', 'LineWidth', 2, 'DisplayName', '1/2 log(cosh(2 \cdot))')
xlabel('x')
legend
grid on
axis equal
hold off
```



要素画像の数

```
nDims = prod(szBlk);
nAtoms = ceil(redundancyRatio*nDims);
```

辞書 Φ の初期化

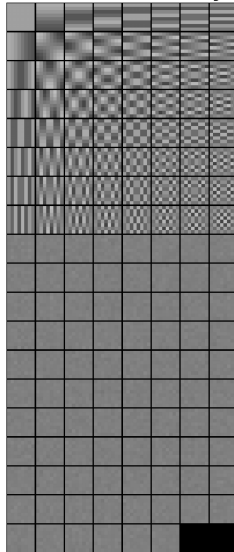
- 二次元離散コサイン変換
- ランダム

```
Phi_rica = randn(nDims,nAtoms);
Phi_rica = Phi_rica/norm(Phi_rica,'fro');
for iAtom = 1:nDims
    delta = zeros(szBlk);
    delta(iAtom) = 1;
    Phi_rica(:,iAtom) = reshape(idct2(delta),nDims,1);
end
```

要素ベクトルを要素画像に変換

```
atomicImagesRica = zeros(szBlk(1),szBlk(2),nAtoms);
for iAtom = 1:nAtoms
    atomicImagesRica(:, :, iAtom) =
    reshape(Phi_rica(:,iAtom),szBlk(1),szBlk(2));
end
figure
montage(imresize(atomicImagesRica,8,'nearest')+.5,'BorderSize',[2 2],'Size',
[ceil(nAtoms/8) 8])
title('Atomic images of initial dictionary (DCT & random)')
```


Atomic images of initial dictionary (DCT & random)



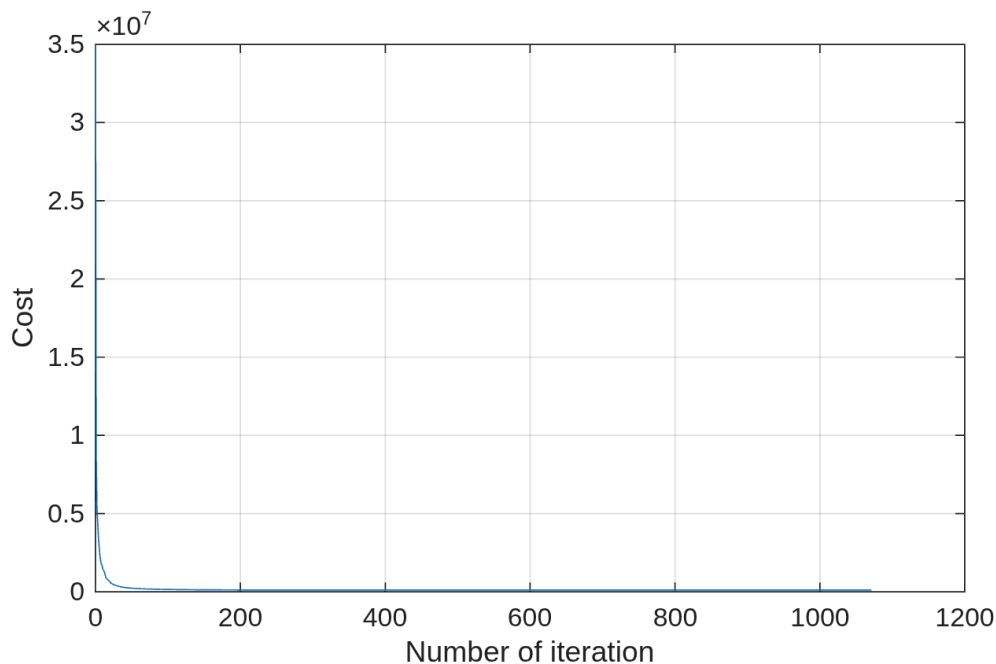
再構成 **ICA** オブジェクトの作成

PCA に合わせて予め零平均化したデータで学習

```
model = rica(Y.', nAtoms, ...  
            'IterationLimit', nItersRica, ...  
            'ContrastFcn', 'logcosh', ...  
            'InitialTransformWeight', Phi_rica, ...  
            'Lambda', 1/(2*alpha));
```

コスト評価のグラフ

```
info = model.FitInfo;  
figure  
plot(info.Iteration, info.Objective)  
xlabel('Number of iteration')  
ylabel('Cost')  
grid on
```



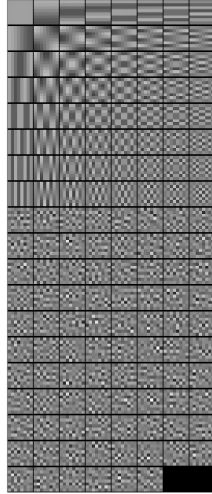
要素ベクトルを要素画像に変換

```
Phi_rica = model.TransformWeights;  
atomicImagesRica = zeros(szBlk(1),szBlk(2),nAtoms);  
for iAtom = 1:nAtoms  
    atomicImagesRica(:, :, iAtom) =  
    reshape(Phi_rica(:, iAtom), szBlk(1), szBlk(2));  
end
```

要素画像の表示（辞書）

```
figure  
montage(imresize(atomicImagesRica, 8, 'nearest')+.5, 'BorderSize', [2 2], 'Size',  
[ceil(nAtoms/8) 8])  
title('Atomic images of RICA')
```

Atomic images of RICA



ブロック **RICA** による合成処理とその随伴処理の定義

```
syn_blkrica = @(x) col2im(Phi_rica*x,szBlk,szOrg,"distinct");  
adj_blkrica = @(y) Phi_rica.'*im2col(y,szBlk,"distinct");
```

随伴関係の確認

```
x = adj_blkrica(y);  
v = randn(size(x));  
u = syn_blkrica(v);  
assert(abs(dot(y(:),u(:))-dot(x(:),v(:)))<1e-9)
```

K-特異値分解

パラメータ設定

- 繰り返し回数 (Number of iterations)

```
% Number of iterations  
nItersKsvd = 5e3;
```

問題設定 (**Problem setting**):

$$\{\hat{\Phi}, \{\hat{\mathbf{x}}_b\}\} = \arg \min_{\{\Phi, \{\mathbf{x}_b\}\}} \frac{1}{2S} \sum_{b=1}^S \|\mathbf{y}_b - \Phi \hat{\mathbf{x}}_b\|_2^2, \quad \text{s.t. } \forall b, \|\mathbf{x}_b\|_0 \leq K$$

アルゴリズム:

スパース近似ステップと辞書更新ステップを繰り返す。

- スパース近似ステップ

$$\hat{\mathbf{x}}_b = \arg \min_{\mathbf{x}} \frac{1}{2} \|\mathbf{y}_b - \hat{\Phi} \mathbf{x}\|_2^2 \quad \text{s.t.} \quad \|\mathbf{x}\|_0 \leq K$$

- 辞書更新ステップ

$$\hat{\Phi} = \arg \min_{\Phi} \frac{1}{2S} \sum_{b=1}^S \|\mathbf{y}_b - \Phi \hat{\mathbf{x}}_b\|_2^2 = \arg \min_{\Phi} \frac{1}{2S} \left\| \left(\mathbf{Y} - \sum_{p \neq k} \phi_p \hat{\mathbf{x}}_{p,:} \right) - \phi_k \hat{\mathbf{x}}_{k,:} \right\|_F^2$$

係数の数

```
nCoefsKsvd = max(floor(sparsityRatio*nDims),1);
```

辞書 Φ の初期化

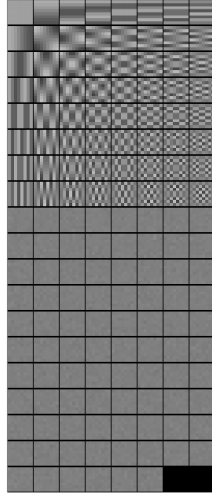
- 二変量離散コサイン変換
- ランダム

```
Phi_ksvd = randn(nDims,nAtoms);
Phi_ksvd = Phi_ksvd/norm(Phi_ksvd,'fro');
for iAtom = 1:nDims
    delta = zeros(szBlk);
    delta(iAtom) = 1;
    Phi_ksvd(:,iAtom) = reshape(idct2(delta),nDims,1);
end
```

要素ベクトルを要素画像に変換

```
atomicImagesKsvd = zeros(szBlk(1),szBlk(2),nAtoms);
for iAtom = 1:nAtoms
    atomicImagesKsvd(:,:,iAtom) =
    reshape(Phi_ksvd(:,iAtom),szBlk(1),szBlk(2));
end
figure
montage(imresize(atomicImagesKsvd,8,'nearest')+.5,'BorderSize',[2 2],'Size',
[ceil(nAtoms/8) 8])
title('Atomic images of initial dictionary (DCT & random)')
```

nic images of initial dictionary (DCT & ran



スパース近似ステップと辞書更新ステップの繰り返し

- スパース近似：直交マッチング追跡 (OMP)
- 辞書更新：特異値分解(SVD)と1-ランク近似

辞書更新の内容

1. $k \leftarrow 1$
2. 誤差行列 $\mathbf{E}_k$ を定義： $\mathbf{E}_k := \mathbf{Y} - \sum_{p \neq k} \phi_p \hat{\mathbf{X}}_{p,:}$
3. データ行 $\hat{\mathbf{X}}_{k,:}$ の非零値を抽出する行列 $\mathbf{\Omega}_k$ を定義： $\hat{\mathbf{X}}_{k,:}^R = \hat{\mathbf{X}}_{k,:} \mathbf{\Omega}_k \Leftrightarrow \hat{\mathbf{X}}_{k,:}^R \mathbf{\Omega}_k^T = \hat{\mathbf{X}}_{k,:}$
4. 誤差行列 $\mathbf{E}_k$ を行列 $\mathbf{\Omega}_k$ で縮退： $\mathbf{E}_k^R := \mathbf{E}_k \mathbf{\Omega}_k$
5. 縮退した誤差行列 $\mathbf{E}_k^R$ を特異値分解： $\mathbf{E}_k^R = \mathbf{U} \mathbf{S} \mathbf{V}^T = (\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_r) \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_r) (\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_r)^T$
6. 要素ベクトル ϕ_k を更新： $\mathbf{k} \leftarrow \mathbf{u}_1$
7. データ行 $\hat{\mathbf{X}}_{k,:}$ を更新： $\hat{\mathbf{X}}_{k,:} \leftarrow \sigma_1 \mathbf{v}_1^T$
8. $k \leftarrow k + 1$
9. $k \leq N$ ならば 2. へ $k > N$ ならば終了

ただし、 σ_1 を最大特異値とする。

交互ステップの繰り返し計算

PCA に合わせて予め零平均化したデータで学習

```
cost = zeros(1,nItersKsvd);
nSamples = size(Y,2);
for iIter = 1:nItersKsvd
    X = zeros(nAtoms,nSamples);
```

```

% Sparse approximation
for iSample = 1:nSamples
    y_ = Y(:,iSample);
    x = omp(y_,Phi_ksvd,nCoefsKSvd);
    X(:,iSample) = x;
end
% Dictionary update
% R is kept as the running residual Y-Phi_ksvd*X so that each atom's
% error matrix can be recovered by adding back its own contribution,
% avoiding an O(nAtoms) setdiff and a full (nAtoms-1)-column matrix
% product on every iteration.
R = Y - Phi_ksvd*X;
for iAtom = 1:nAtoms
    xk = X(iAtom,:);
    suppk = find(xk);
    %
    if ~isempty(suppk)
        Ekred = R(:,suppk) + Phi_ksvd(:,iAtom)*xk(suppk);
        [U,S,V] = svd(Ekred,'econ');
        ak = U(:,1);
        xkred = S(1,1)*V(:,1)';
        %
        Phi_ksvd(:,iAtom) = ak;
        X(iAtom,suppk) = xkred;
        R(:,suppk) = Ekred - ak*xkred;
    end
end
cost(iIter) = (norm(Y-Phi_ksvd*X,'fro')^2)/(2*nSamples);
end

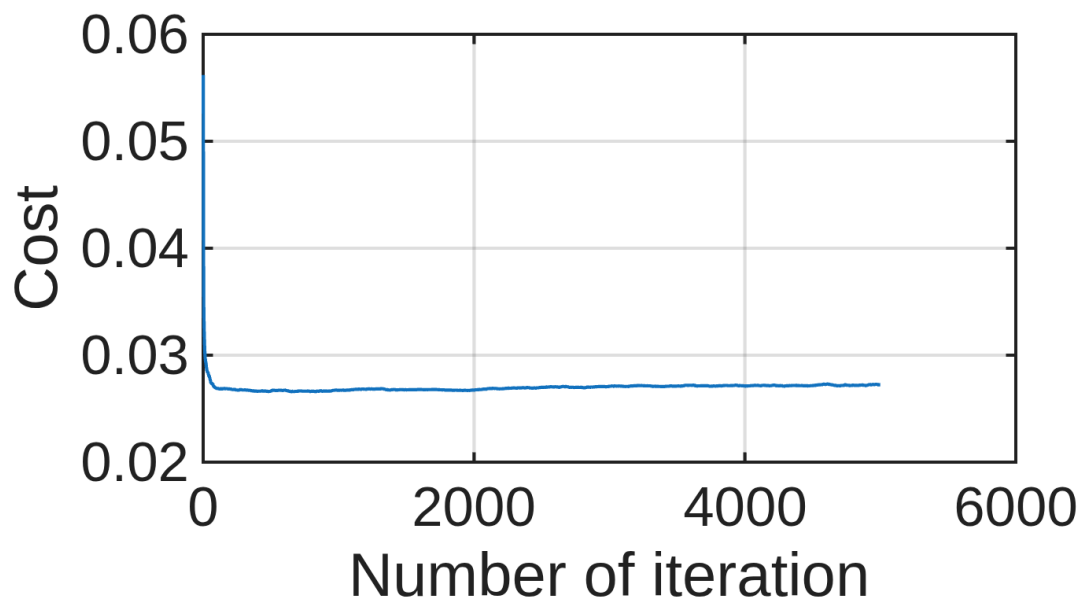
```

コスト評価のグラフ

```

figure
plot(cost)
xlabel('Number of iteration')
ylabel('Cost')
grid on

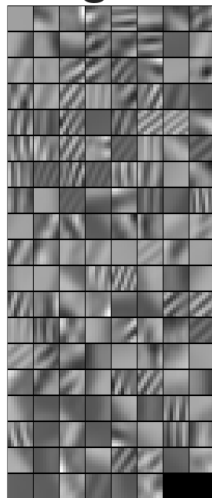
```



要素ベクトルを要素画像に変換

```
atomicImagesKsvd = zeros(szBlk(1),szBlk(2),nAtoms);
for iAtom = 1:nAtoms
    atomicImagesKsvd(:, :, iAtom) =
    reshape(Phi_ksvd(:, iAtom), szBlk(1), szBlk(2));
end
figure
montage(imresize(atomicImagesKsvd, 8, 'nearest')+.5, 'BorderSize', [2 2], 'Size',
[ceil(nAtoms/8) 8])
title('Atomic images of K-SVD')
```

Atomic images of K-SVD



```
syn_blkksvd = @(x) col2im(Phi_ksvd*x,szBlk,szOrg,"distinct");
adj_blkksvd = @(y) Phi_ksvd.'*im2col(y,szBlk,"distinct");
```

随伴関係の確認

```
x = adj_blkksvd(y);
v = randn(size(x));
u = syn_blkksvd(v);
assert(abs(dot(y(:),u(:))-dot(x(:),v(:)))<1e-9)
```

2 変量ラティス構造冗長フィルタバンク

例として，（偶対称チャンネルと奇対称チャンネルが等しい）偶数チャンネル、偶数のポリフェーズ次数をもつタイプ I 非分離冗長重複変換(NSOLT)

$$\mathbf{E}(z_v, z_h) = \left(\prod_{n_h=1}^{\nu_h/2} \mathbf{V}_{2n_h}^{(h)} \bar{\mathbf{Q}}(z_h) \mathbf{V}_{2n_h-1}^{(h)} \mathbf{Q}(z_h) \right) \left(\prod_{n_v=1}^{\nu_v/2} \mathbf{V}_{2n_v}^{(v)} \bar{\mathbf{Q}}(z_v) \mathbf{V}_{2n_v-1}^{(v)} \mathbf{Q}(z_v) \right) \mathbf{V}_0 \mathbf{E}_0,$$

$$\mathbf{R}(z_v, z_h) = \mathbf{E}^T(z_v^{-1}, z_h^{-1}),$$

を採用する．ただし，

- $\mathbf{E}(z_v, z_h)$: 分析フィルタバンクの Type-I ポリフェーズ行列
- $\mathbf{R}(z_v, z_h)$: 合成フィルタバンクの Type-II ポリフェーズ行列
- $z_d \in \mathbb{C}, d \in \{v, h\}$: Z-変換の変数
- $\nu_d \in \mathbb{N}, d \in \{v, h\}$: 方向 d のポリフェーズ次数(重複ブロック数)

$$\mathbf{V}_0 = \begin{pmatrix} \mathbf{W}_0 & \mathbf{O} \\ \mathbf{O} & \mathbf{U}_0 \end{pmatrix} \begin{pmatrix} \mathbf{I}_{M/2} \\ \mathbf{O} \\ \mathbf{I}_{M/2} \\ \mathbf{O} \end{pmatrix} \in \mathbb{R}^{P \times M}, \mathbf{V}_n^{(d)} = \begin{pmatrix} \mathbf{I}_{P/2} & \mathbf{O} \\ \mathbf{O} & \mathbf{U}_n^{(d)} \end{pmatrix} \in \mathbb{R}^{P \times P}, d \in \{v, h\}, \mathbf{W}_0, \mathbf{U}_0, \mathbf{U}_n^{(d)} \in \mathbb{R}^{P/2 \times P/2} \text{ は直交行列}$$

$$\mathbf{Q}(z) = \mathbf{B}_P \begin{pmatrix} \mathbf{I}_{P/2} & \mathbf{O} \\ \mathbf{O} & z^{-1} \mathbf{I}_{P/2} \end{pmatrix} \mathbf{B}_P, \bar{\mathbf{Q}}(z) = \mathbf{B}_P \begin{pmatrix} z \mathbf{I}_{P/2} & \mathbf{O} \\ \mathbf{O} & \mathbf{I}_{P/2} \end{pmatrix} \mathbf{B}_P, \mathbf{B}_P = \frac{1}{\sqrt{2}} \begin{pmatrix} \mathbf{I}_{P/2} & \mathbf{I}_{P/2} \\ \mathbf{I}_{P/2} & -\mathbf{I}_{P/2} \end{pmatrix}$$

【References】

- [Overview of Filter Banks - MATLAB & Simulink - MathWorks 日本](#)
- MATLAB SaivDr Package: <https://github.com/msiplab/SaivDr>
- S. Muramatsu, K. Furuya and N. Yuki, "Multidimensional Nonseparable Oversampled Lapped Transforms: Theory and Design," in IEEE Transactions on Signal Processing, vol. 65, no. 5, pp. 1251-1264, 1 March 1, 2017, doi: 10.1109/TSP.2016.2633240.

- S. Muramatsu, T. Kobayashi, M. Hiki and H. Kikuchi, "Boundary Operation of 2-D Nonseparable Linear-Phase Paraunitary Filter Banks," in IEEE Transactions on Image Processing, vol. 21, no. 4, pp. 2314-2318, April 2012, doi: 10.1109/TIP.2011.2181527.
- S. Muramatsu, M. Ishii and Z. Chen, "Efficient parameter optimization for example-based design of nonseparable oversampled lapped transform," 2016 IEEE International Conference on Image Processing (ICIP), Phoenix, AZ, 2016, pp. 3618-3622, doi: 10.1109/ICIP.2016.7533034.
- Furuya, K., Hara, S., Seino, K., & Muramatsu, S. (2016). Boundary operation of 2D non-separable oversampled lapped transforms. *APSIPA Transactions on Signal and Information Processing*, 5, E9. doi:10.1017/ATSIP.2016.3.

2次元画像の階層的分析

$R_M^P(\tau)$ をツリーレベル τ の階層構造フィルタバンクの冗長度とすると、

$$R_M^P(\tau) = \begin{cases} (P-1)\tau + 1, & M = 1, \\ \frac{P-1}{M-1} - \frac{P-M}{(M-1)M^\tau}, & M \geq 2. \end{cases}$$

となる。

構成パラメータ設定

```
%%{
% Decimation factor (Strides)
decFactor = [2 2]; % [μv μh]

% Number of channels ( sum(nChannels) >= prod(decFactors) )
nChannels = [4 4]; % [Ps Pa] (Ps=Pa)

% Number of tree levels
nLevels = 4;

% Polyphase Order
ppOrder = [4 4];
%%}

%{
% Decimation factor (Strides)
decFactor = [4 4]; % [μv μh]

% Number of channels ( sum(nChannels) >= prod(decFactors) )
nChannels = [13 13]; % [Ps Pa] (Ps=Pa)

% Number of tree levels
nLevels = 2;

% Polyphase Order
ppOrder = [2 2];
```

```

%}

%{
% Decimation factor (Strides)
decFactor = [8 8]; % [ $\mu_v$   $\mu_h$ ]

% Number of channels ( sum(nChannels) >= prod(decFactors) )
nChannels = [53 53]; % [Ps Pa] (Ps=Pa)

% Number of tree levels
nLevels = 1;

% Polyphase Order
ppOrder = [2 2];
%}

% Redundancy
P = sum(nChannels);
M = prod(decFactor);
redundancyNsolt = ...
    (prod(decFactor)==1)*((P-1)*nLevels+1) + ...
    (prod(decFactor)>1)*((P-1)/(M-1)-(P-M)/((M-1)*M^nLevels))

redundancyNsolt =
2.3281

```

```

assert(redundancyNsolt<redundancyRatio)

```

$$L_v \times L_h = \left(\mu_v^\tau + \nu_v \frac{\mu_v(\mu_v^\tau - 1)}{\mu_v - 1} \right) \times \left(\mu_h^\tau + \nu_h \frac{\mu_h(\mu_h^\tau - 1)}{\mu_h - 1} \right)$$

```

% Filter size [ Ly Lx ]
maxDecFactor = decFactor.^nLevels;
szFilters = maxDecFactor + ppOrder.*decFactor.*(maxDecFactor-1)./
(decFactor-1)

```

```

szFilters = 1x2
    136    136

```

```

% Patch size for training
szPatchTrn = maxDecFactor.*ceil(szFilters./maxDecFactor) % > [ Ly Lx ]

```

```

szPatchTrn = 1x2
    144    144

```

```

%szPatchTrn = 2.^nextpow2(szFilters) % > [ Ly Lx ]
assert(all(szPatchTrn>szFilters))

```

```

% Number of patches per image
nSubImgs = floor(nPatches*prod(szBlk./szPatchTrn))

```

```
nSubImgs =  
47
```

```
assert(nSubImgs > 0)
```

```
% No DC-leakage  
noDcLeakage = true
```

```
noDcLeakage =  
logical  
1
```

辞書の設定

```
if exist("../data/"+nsoltDic+".mat","file")  
    S = load("../data/"+nsoltDic);  
    analysisnet = S.analysisnet;  
    synthesisnet = S.synthesisnet;  
    nLevels_ = extractnumlevels(analysisnet);  
    decFactor_ = extractdecfactor(analysisnet);  
    nChannels_ = extractnumchannels(analysisnet);  
  
    assert(nLevels==nLevels_)  
    assert(all(decFactor==decFactor_))  
    assert(all(nChannels==nChannels_))  
else  
    % Number of iterations  
    nItersNsolt = 10;  
  
    % Standard deviation of initial angles  
    stdInitAng = 1e-1; %pi/6;  
  
    % Mini batch size  
    miniBatchSize = 10;  
  
    % Number of Epochs (1 Epoch = nSubImgs/miniBatchSize iterations)  
    maxEpochs = 30;  
  
    % Number of iterations  
    maxIters = nSubImgs/miniBatchSize * maxEpochs  
  
    % Training options  
    opts = trainingOptions('sgdm', ... % Stochastic gradient descent w/  
momentum  
        ... 'Momentum', 0.9000, ...  
        'InitialLearnRate', 5.0e-03, ...  
        ... 'LearnRateScheduleSettings', 'none', ...  
        'L2Regularization', 0.0, ... 1.0e-04, ...  
        ... 'GradientThresholdMethod', 'l2norm', ...  
        ... 'GradientThreshold', Inf, ...  
        'MaxEpochs', maxEpochs, ... 30, ...
```

```

'MiniBatchSize',miniBatchSize,...128,...
'Verbose',1,...
...'VerboseFrequency',50,...
...'ValidationData',[],...
...'ValidationFrequency',50,...
...'ValidationPatience',Inf,...
...'Shuffle','once',...
...'CheckpointPath','',...
...'ExecutionEnvironment','auto',...
...'WorkerLoad',[],...
...'OutputFcn',[],...
'Plots','none',...'training-progress',...
...'SequenceLength','longest',...
...'SequencePaddingValue',0,...
...'SequencePaddingDirection','right',...
...'DispatchInBackground',0,...
'ResetInputNormalization',0);...1

```

層構造の構築

```

import saivdr.dcn.*
analysislgraph = fcn_creatensoltlgraph2d([],...
'InputSize',szPatchTrn,...
'NumberOfChannels',nChannels,...
'DecimationFactor',decFactor,...
'PolyPhaseOrder',ppOrder,...
'NumberOfLevels',nLevels,...
'NumberOfVanishingMoments',noDcLeakage,...
'Mode','Analyzer');
synthesislgraph = fcn_creatensoltlgraph2d([],...
'InputSize',szPatchTrn,...
'NumberOfChannels',nChannels,...
'DecimationFactor',decFactor,...
'PolyPhaseOrder',ppOrder,...
'NumberOfLevels',nLevels,...
'NumberOfVanishingMoments',noDcLeakage,...
'Mode','Synthesizer');

figure
subplot(1,2,1)
plot(analysislgraph)
title('Analysis NSOLT')
subplot(1,2,2)
plot(synthesislgraph)
title('Synthesis NSOLT')

% Construction of deep learning network.
synthesisnet = dlnetwork(synthesislgraph);

% Initialize
nLearnables = height(synthesisnet.Learnables);

```

```

for iLearnable = 1:nLearnables
    if synthesisnet.Learnables.Parameter(iLearnable)=="Angles"
        layerName = synthesisnet.Learnables.Layer(iLearnable);
        synthesisnet.Learnables.Value(iLearnable) = ...
            cellfun(@(x) x+stdInitAng*randn(size(x)), ...

synthesisnet.Learnables.Value(iLearnable), 'UniformOutput', false);
    end
end

% Copy the synthesizer's parameters to the analyzer
synthesislgraph = layerGraph(synthesisnet);
analysislgraph = fcn_cpparamssyn2ana(analysislgraph, synthesislgraph);
analysisnet = dlnetwork(analysislgraph);

```

随伴関係（完全再構成）の確認

NSOLT はパーセバルタイト性を満たす.

```

nOutputs = nLevels+1;
x = rand(szPatchTrn, 'single');
s = cell(1, nOutputs);
dlx = dlarray(x, 'SSCB'); % Deep learning array (SSCB:
Spatial, Spatial, Channel, Batch)
[s{1:nOutputs}] = analysisnet.predict(dlx);
dly = synthesisnet.predict(s{:});
display("MSE: " + num2str(mse(dlx, dly)))

```

要素画像の初期状態

```

import saivdr.dcn.*
figure
atomicimshow(synthesisnet, [], 2^(nLevels-1))
title('Atomic images of initial NSOLT')

```

訓練画像の準備

画像データストアからランダムにパッチを抽出

PCA に合わせて予め零平均化したデータで学習

```

imds = imageDatastore(file_yorg, "ReadFcn", @(x)
meansubtract(im2single(imread(x))));
patchds =
randomPatchExtractionDatastore(imds, imds, szPatchTrn, 'PatchesPerImage', nSubImgs);
figure
minibatch = preview(patchds);
responses = minibatch.ResponseImage;
responses = cellfun(@(x) x + 0.5, responses, 'UniformOutput', false);
figure
montage(responses, 'Size', [2 4]);

```

畳み込み辞書学習

問題設定:

$$\{\hat{\theta}, \{\hat{\mathbf{x}}_n\}\} = \arg \min_{\{\theta, \{\mathbf{x}_n\}\}} \frac{1}{2S} \sum_{n=1}^S \|\mathbf{y}_n - \mathbf{D}_{\theta} \hat{\mathbf{x}}_n\|_2^2, \quad \text{s.t. } \forall n, \|\mathbf{x}_n\|_0 \leq K,$$

ただし, $\mathbf{D}_{\theta}$ は設計パラメータベクトル θ をもつ畳み込み辞書.

アルゴリズム:

スパース近似ステップと辞書更新ステップを繰り返す.

- スパース近似ステップ

$$\hat{\mathbf{x}}_n = \arg \min_{\mathbf{x}_n} \frac{1}{2} \|\mathbf{y}_n - \hat{\mathbf{D}} \mathbf{x}_n\|_2^2 \quad \text{s.t. } \|\mathbf{x}_n\|_0 \leq K$$

- 辞書更新ステップ

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{2S} \sum_{n=1}^S \|\mathbf{y}_n - \mathbf{D}_{\theta} \hat{\mathbf{x}}_n\|_2^2$$

$$\hat{\mathbf{D}} = \mathbf{D}_{\hat{\theta}}$$

採用するスパース近似と辞書更新の手法:

- スパース近似: (正規化なし) 繰り返しハード閾値処理(IHT)
- 辞書更新: モーメント付き確率的勾配降下法(SGD)

```
% Check if IHT works for dlarray
%x = dlarray(randn(szPatchTrn, 'single'), 'SSCB');
%[y,coefs{1:nOutputs}] = iht(x,analysisnet,synthesisnet,sparsityRatio);
```

辞書学習の繰り返し計算

```
import saivdr.dcn.*
%profile on
for iIter = 1:nItersNsolt

    % Sparse approximation (Applied to produce an object of
    TransformedDatastore)
    coefimgds = transform(patchds, @(x)
    iht4patchds(x,analysisnet,synthesisnet,sparsityRatio));
```

```

% Synthesis dictionary update
trainlgraph = synthesislgraph.replaceLayer('Lv1_Out', ...
    regressionLayer('Name', 'Lv1_Out'));
trainednet = trainNetwork(coefimgds, trainlgraph, opts);

% Analysis dictionary update (Copy parameters from synthesizer to
analyzer)
trainedlgraph = layerGraph(trainednet);
analysislgraph = fcn_cpparamssyn2ana(analysislgraph, trainedlgraph);
analysisnet = dlnetwork(analysislgraph);

% Check the adjoint relation (perfect reconstruction)

checkadjointrelation(analysislgraph, trainedlgraph, nLevels, szPatchTrn);

% Replace layer
synthesislgraph = trainedlgraph.replaceLayer('Lv1_Out', ...
    nsoltIdentityLayer('Name', 'Lv1_Out'));
synthesisnet = dlnetwork(synthesislgraph);

end
%profile off
%profile viewer

```

訓練ネットワークの保存

```

import saivdr.dcn.*
synthesislgraph = layerGraph(synthesisnet);
analysislgraph = fcn_cpparamssyn2ana(analysislgraph, synthesislgraph);
analysisnet = dlnetwork(analysislgraph);
save(sprintf('..../results/
nsoltdictionary_%s', datetime('now', 'Format', 'yyyyMMddHHmmssSSS')), 'analysisne
t', 'synthesisnet', 'nLevels')
end

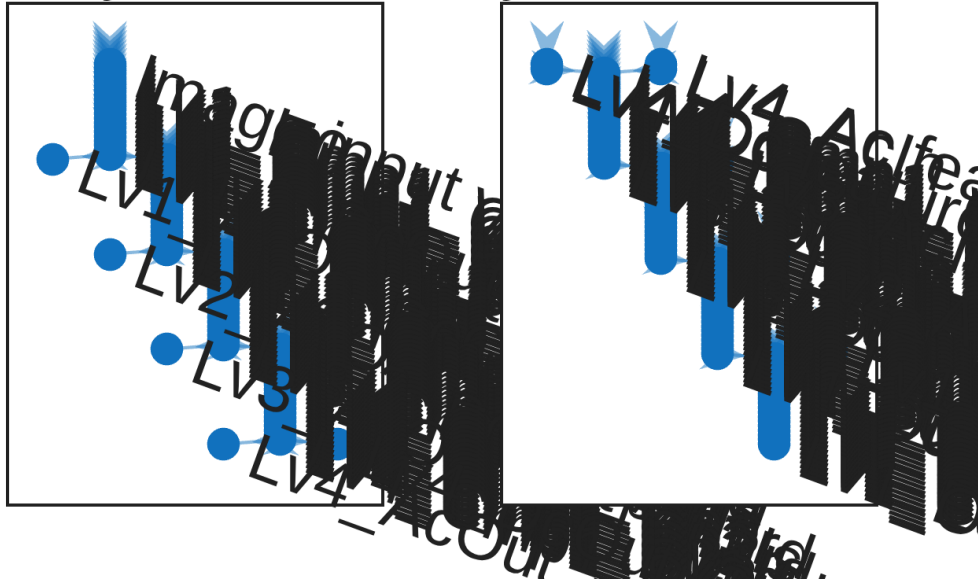
```

```

maxIters =
141

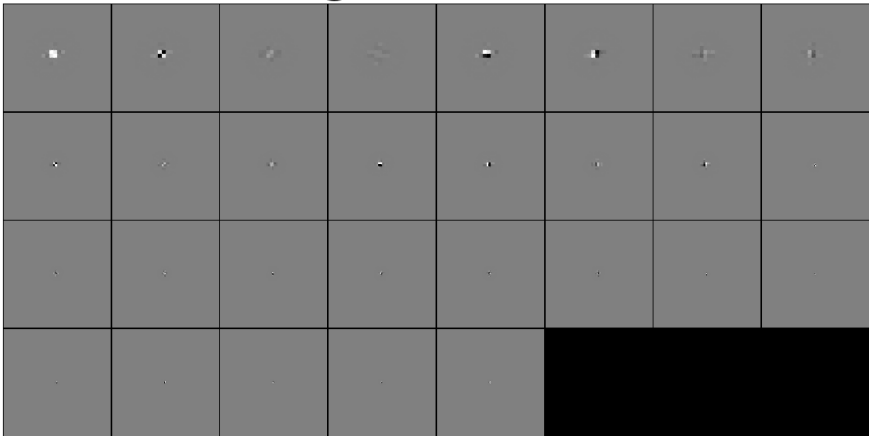
```

Analysis NSOLT Synthesis NSOLT



Copy angles from Lv1_Cmp1_V0~ to Lv1_Cmp1_V0
 Copy angles from Lv1_Cmp1_Vh1~ to Lv1_Cmp1_Vh1
 Copy angles from Lv1_Cmp1_Vh2~ to Lv1_Cmp1_Vh2
 Copy angles from Lv1_Cmp1_Vh3~ to Lv1_Cmp1_Vh3
 Copy angles from Lv1_Cmp1_Vh4~ to Lv1_Cmp1_Vh4
 Copy angles from Lv1_Cmp1_Vv1~ to Lv1_Cmp1_Vv1
 Copy angles from Lv1_Cmp1_Vv2~ to Lv1_Cmp1_Vv2
 Copy angles from Lv1_Cmp1_Vv3~ to Lv1_Cmp1_Vv3
 Copy angles from Lv1_Cmp1_Vv4~ to Lv1_Cmp1_Vv4
 Copy angles from Lv2_Cmp1_V0~ to Lv2_Cmp1_V0
 Copy angles from Lv2_Cmp1_Vh1~ to Lv2_Cmp1_Vh1
 Copy angles from Lv2_Cmp1_Vh2~ to Lv2_Cmp1_Vh2
 Copy angles from Lv2_Cmp1_Vh3~ to Lv2_Cmp1_Vh3
 Copy angles from Lv2_Cmp1_Vh4~ to Lv2_Cmp1_Vh4
 Copy angles from Lv2_Cmp1_Vv1~ to Lv2_Cmp1_Vv1
 Copy angles from Lv2_Cmp1_Vv2~ to Lv2_Cmp1_Vv2
 Copy angles from Lv2_Cmp1_Vv3~ to Lv2_Cmp1_Vv3
 Copy angles from Lv2_Cmp1_Vv4~ to Lv2_Cmp1_Vv4
 Copy angles from Lv3_Cmp1_V0~ to Lv3_Cmp1_V0
 Copy angles from Lv3_Cmp1_Vh1~ to Lv3_Cmp1_Vh1
 Copy angles from Lv3_Cmp1_Vh2~ to Lv3_Cmp1_Vh2
 Copy angles from Lv3_Cmp1_Vh3~ to Lv3_Cmp1_Vh3
 Copy angles from Lv3_Cmp1_Vh4~ to Lv3_Cmp1_Vh4
 Copy angles from Lv3_Cmp1_Vv1~ to Lv3_Cmp1_Vv1
 Copy angles from Lv3_Cmp1_Vv2~ to Lv3_Cmp1_Vv2
 Copy angles from Lv3_Cmp1_Vv3~ to Lv3_Cmp1_Vv3
 Copy angles from Lv3_Cmp1_Vv4~ to Lv3_Cmp1_Vv4
 Copy angles from Lv4_Cmp1_V0~ to Lv4_Cmp1_V0
 Copy angles from Lv4_Cmp1_Vh1~ to Lv4_Cmp1_Vh1
 Copy angles from Lv4_Cmp1_Vh2~ to Lv4_Cmp1_Vh2
 Copy angles from Lv4_Cmp1_Vh3~ to Lv4_Cmp1_Vh3
 Copy angles from Lv4_Cmp1_Vh4~ to Lv4_Cmp1_Vh4
 Copy angles from Lv4_Cmp1_Vv1~ to Lv4_Cmp1_Vv1
 Copy angles from Lv4_Cmp1_Vv2~ to Lv4_Cmp1_Vv2
 Copy angles from Lv4_Cmp1_Vv3~ to Lv4_Cmp1_Vv3
 Copy angles from Lv4_Cmp1_Vv4~ to Lv4_Cmp1_Vv4
 "MSE: 3.2498e-10"

Atomic images of initial NSOLT



単一の GPU で学習中。

エポック	反復	経過時間 (h h : mm : s s)	ミニバッチ RMSE	ミニバッチ損失	基本学
1	1	00 : 00 : 05	6.46	20.8	0.0
13	50	00 : 01 : 18	5.94	17.6	0.0
25	100	00 : 02 : 32	5.60	15.7	0.0
30	120	00 : 03 : 02	5.43	14.7	0.0

学習終了：最大エポック数が完了しました。
Copy angles from Lv1_Cmp1_V0~ to Lv1_Cmp1_V0
Copy angles from Lv1_Cmp1_Vh1~ to Lv1_Cmp1_Vh1
Copy angles from Lv1_Cmp1_Vh2~ to Lv1_Cmp1_Vh2
Copy angles from Lv1_Cmp1_Vh3~ to Lv1_Cmp1_Vh3
Copy angles from Lv1_Cmp1_Vh4~ to Lv1_Cmp1_Vh4

Copy angles from Lv1_Cmp1_Vv1~ to Lv1_Cmp1_Vv1
 Copy angles from Lv1_Cmp1_Vv2~ to Lv1_Cmp1_Vv2
 Copy angles from Lv1_Cmp1_Vv3~ to Lv1_Cmp1_Vv3
 Copy angles from Lv1_Cmp1_Vv4~ to Lv1_Cmp1_Vv4
 Copy angles from Lv2_Cmp1_V0~ to Lv2_Cmp1_V0
 Copy angles from Lv2_Cmp1_Vh1~ to Lv2_Cmp1_Vh1
 Copy angles from Lv2_Cmp1_Vh2~ to Lv2_Cmp1_Vh2
 Copy angles from Lv2_Cmp1_Vh3~ to Lv2_Cmp1_Vh3
 Copy angles from Lv2_Cmp1_Vh4~ to Lv2_Cmp1_Vh4
 Copy angles from Lv2_Cmp1_Vv1~ to Lv2_Cmp1_Vv1
 Copy angles from Lv2_Cmp1_Vv2~ to Lv2_Cmp1_Vv2
 Copy angles from Lv2_Cmp1_Vv3~ to Lv2_Cmp1_Vv3
 Copy angles from Lv2_Cmp1_Vv4~ to Lv2_Cmp1_Vv4
 Copy angles from Lv3_Cmp1_V0~ to Lv3_Cmp1_V0
 Copy angles from Lv3_Cmp1_Vh1~ to Lv3_Cmp1_Vh1
 Copy angles from Lv3_Cmp1_Vh2~ to Lv3_Cmp1_Vh2
 Copy angles from Lv3_Cmp1_Vh3~ to Lv3_Cmp1_Vh3
 Copy angles from Lv3_Cmp1_Vh4~ to Lv3_Cmp1_Vh4
 Copy angles from Lv3_Cmp1_Vv1~ to Lv3_Cmp1_Vv1
 Copy angles from Lv3_Cmp1_Vv2~ to Lv3_Cmp1_Vv2
 Copy angles from Lv3_Cmp1_Vv3~ to Lv3_Cmp1_Vv3
 Copy angles from Lv3_Cmp1_Vv4~ to Lv3_Cmp1_Vv4
 Copy angles from Lv4_Cmp1_V0~ to Lv4_Cmp1_V0
 Copy angles from Lv4_Cmp1_Vh1~ to Lv4_Cmp1_Vh1
 Copy angles from Lv4_Cmp1_Vh2~ to Lv4_Cmp1_Vh2
 Copy angles from Lv4_Cmp1_Vh3~ to Lv4_Cmp1_Vh3
 Copy angles from Lv4_Cmp1_Vh4~ to Lv4_Cmp1_Vh4
 Copy angles from Lv4_Cmp1_Vv1~ to Lv4_Cmp1_Vv1
 Copy angles from Lv4_Cmp1_Vv2~ to Lv4_Cmp1_Vv2
 Copy angles from Lv4_Cmp1_Vv3~ to Lv4_Cmp1_Vv3
 Copy angles from Lv4_Cmp1_Vv4~ to Lv4_Cmp1_Vv4

"MSE: 3.3547e-14"

単一の GPU で学習中。

エポック	反復	経過時間 (h h : mm : s s)	ミニバッチ RMSE	ミニバッチ損失	基本学
1	1	00 : 00 : 04	5.48	15.0	0.0
13	50	00 : 01 : 18	4.97	12.4	0.0
25	100	00 : 02 : 32	5.20	13.5	0.0
30	120	00 : 03 : 02	5.05	12.7	0.0

学習終了: 最大エポック数が完了しました。

Copy angles from Lv1_Cmp1_V0~ to Lv1_Cmp1_V0
 Copy angles from Lv1_Cmp1_Vh1~ to Lv1_Cmp1_Vh1
 Copy angles from Lv1_Cmp1_Vh2~ to Lv1_Cmp1_Vh2
 Copy angles from Lv1_Cmp1_Vh3~ to Lv1_Cmp1_Vh3
 Copy angles from Lv1_Cmp1_Vh4~ to Lv1_Cmp1_Vh4
 Copy angles from Lv1_Cmp1_Vv1~ to Lv1_Cmp1_Vv1
 Copy angles from Lv1_Cmp1_Vv2~ to Lv1_Cmp1_Vv2
 Copy angles from Lv1_Cmp1_Vv3~ to Lv1_Cmp1_Vv3
 Copy angles from Lv1_Cmp1_Vv4~ to Lv1_Cmp1_Vv4
 Copy angles from Lv2_Cmp1_V0~ to Lv2_Cmp1_V0
 Copy angles from Lv2_Cmp1_Vh1~ to Lv2_Cmp1_Vh1
 Copy angles from Lv2_Cmp1_Vh2~ to Lv2_Cmp1_Vh2
 Copy angles from Lv2_Cmp1_Vh3~ to Lv2_Cmp1_Vh3
 Copy angles from Lv2_Cmp1_Vh4~ to Lv2_Cmp1_Vh4
 Copy angles from Lv2_Cmp1_Vv1~ to Lv2_Cmp1_Vv1
 Copy angles from Lv2_Cmp1_Vv2~ to Lv2_Cmp1_Vv2
 Copy angles from Lv2_Cmp1_Vv3~ to Lv2_Cmp1_Vv3
 Copy angles from Lv2_Cmp1_Vv4~ to Lv2_Cmp1_Vv4
 Copy angles from Lv3_Cmp1_V0~ to Lv3_Cmp1_V0
 Copy angles from Lv3_Cmp1_Vh1~ to Lv3_Cmp1_Vh1
 Copy angles from Lv3_Cmp1_Vh2~ to Lv3_Cmp1_Vh2

Copy angles from Lv3_Cmp1_Vh3~ to Lv3_Cmp1_Vh3
 Copy angles from Lv3_Cmp1_Vh4~ to Lv3_Cmp1_Vh4
 Copy angles from Lv3_Cmp1_Vv1~ to Lv3_Cmp1_Vv1
 Copy angles from Lv3_Cmp1_Vv2~ to Lv3_Cmp1_Vv2
 Copy angles from Lv3_Cmp1_Vv3~ to Lv3_Cmp1_Vv3
 Copy angles from Lv3_Cmp1_Vv4~ to Lv3_Cmp1_Vv4
 Copy angles from Lv4_Cmp1_V0~ to Lv4_Cmp1_V0
 Copy angles from Lv4_Cmp1_Vh1~ to Lv4_Cmp1_Vh1
 Copy angles from Lv4_Cmp1_Vh2~ to Lv4_Cmp1_Vh2
 Copy angles from Lv4_Cmp1_Vh3~ to Lv4_Cmp1_Vh3
 Copy angles from Lv4_Cmp1_Vh4~ to Lv4_Cmp1_Vh4
 Copy angles from Lv4_Cmp1_Vv1~ to Lv4_Cmp1_Vv1
 Copy angles from Lv4_Cmp1_Vv2~ to Lv4_Cmp1_Vv2
 Copy angles from Lv4_Cmp1_Vv3~ to Lv4_Cmp1_Vv3
 Copy angles from Lv4_Cmp1_Vv4~ to Lv4_Cmp1_Vv4

"MSE: 3.0081e-14"

単一の GPU で学習中。

エポック	反復	経過時間 (h h : mm : s s)	ミニバッチ RMSE	ミニバッチ損失	基本学
1	1	00 : 00 : 04	5.26	13.8	0.0
13	50	00 : 01 : 18	5.03	12.7	0.0
25	100	00 : 02 : 32	4.94	12.2	0.0
30	120	00 : 03 : 02	5.11	13.1	0.0

学習終了: 最大エポック数が完了しました。

Copy angles from Lv1_Cmp1_V0~ to Lv1_Cmp1_V0
 Copy angles from Lv1_Cmp1_Vh1~ to Lv1_Cmp1_Vh1
 Copy angles from Lv1_Cmp1_Vh2~ to Lv1_Cmp1_Vh2
 Copy angles from Lv1_Cmp1_Vh3~ to Lv1_Cmp1_Vh3
 Copy angles from Lv1_Cmp1_Vh4~ to Lv1_Cmp1_Vh4
 Copy angles from Lv1_Cmp1_Vv1~ to Lv1_Cmp1_Vv1
 Copy angles from Lv1_Cmp1_Vv2~ to Lv1_Cmp1_Vv2
 Copy angles from Lv1_Cmp1_Vv3~ to Lv1_Cmp1_Vv3
 Copy angles from Lv1_Cmp1_Vv4~ to Lv1_Cmp1_Vv4
 Copy angles from Lv2_Cmp1_V0~ to Lv2_Cmp1_V0
 Copy angles from Lv2_Cmp1_Vh1~ to Lv2_Cmp1_Vh1
 Copy angles from Lv2_Cmp1_Vh2~ to Lv2_Cmp1_Vh2
 Copy angles from Lv2_Cmp1_Vh3~ to Lv2_Cmp1_Vh3
 Copy angles from Lv2_Cmp1_Vh4~ to Lv2_Cmp1_Vh4
 Copy angles from Lv2_Cmp1_Vv1~ to Lv2_Cmp1_Vv1
 Copy angles from Lv2_Cmp1_Vv2~ to Lv2_Cmp1_Vv2
 Copy angles from Lv2_Cmp1_Vv3~ to Lv2_Cmp1_Vv3
 Copy angles from Lv2_Cmp1_Vv4~ to Lv2_Cmp1_Vv4
 Copy angles from Lv3_Cmp1_V0~ to Lv3_Cmp1_V0
 Copy angles from Lv3_Cmp1_Vh1~ to Lv3_Cmp1_Vh1
 Copy angles from Lv3_Cmp1_Vh2~ to Lv3_Cmp1_Vh2
 Copy angles from Lv3_Cmp1_Vh3~ to Lv3_Cmp1_Vh3
 Copy angles from Lv3_Cmp1_Vh4~ to Lv3_Cmp1_Vh4
 Copy angles from Lv3_Cmp1_Vv1~ to Lv3_Cmp1_Vv1
 Copy angles from Lv3_Cmp1_Vv2~ to Lv3_Cmp1_Vv2
 Copy angles from Lv3_Cmp1_Vv3~ to Lv3_Cmp1_Vv3
 Copy angles from Lv3_Cmp1_Vv4~ to Lv3_Cmp1_Vv4
 Copy angles from Lv4_Cmp1_V0~ to Lv4_Cmp1_V0
 Copy angles from Lv4_Cmp1_Vh1~ to Lv4_Cmp1_Vh1
 Copy angles from Lv4_Cmp1_Vh2~ to Lv4_Cmp1_Vh2
 Copy angles from Lv4_Cmp1_Vh3~ to Lv4_Cmp1_Vh3
 Copy angles from Lv4_Cmp1_Vh4~ to Lv4_Cmp1_Vh4
 Copy angles from Lv4_Cmp1_Vv1~ to Lv4_Cmp1_Vv1
 Copy angles from Lv4_Cmp1_Vv2~ to Lv4_Cmp1_Vv2
 Copy angles from Lv4_Cmp1_Vv3~ to Lv4_Cmp1_Vv3
 Copy angles from Lv4_Cmp1_Vv4~ to Lv4_Cmp1_Vv4

"MSE: 2.9565e-14"

単一の GPU で学習中。

エポック	反復	経過時間 (h h : mm : s s)	ミニバッチ RMSE	ミニバッチ損失	基本学
1	1	00 : 00 : 04	5.06	12.8	0.0
13	50	00 : 01 : 18	5.17	13.4	0.0
25	100	00 : 02 : 32	5.01	12.6	0.0
30	120	00 : 03 : 02	5.02	12.6	0.0

学習終了：最大エポック数が完了しました。

Copy angles from Lv1_Cmp1_V0~ to Lv1_Cmp1_V0
Copy angles from Lv1_Cmp1_Vh1~ to Lv1_Cmp1_Vh1
Copy angles from Lv1_Cmp1_Vh2~ to Lv1_Cmp1_Vh2
Copy angles from Lv1_Cmp1_Vh3~ to Lv1_Cmp1_Vh3
Copy angles from Lv1_Cmp1_Vh4~ to Lv1_Cmp1_Vh4
Copy angles from Lv1_Cmp1_Vv1~ to Lv1_Cmp1_Vv1
Copy angles from Lv1_Cmp1_Vv2~ to Lv1_Cmp1_Vv2
Copy angles from Lv1_Cmp1_Vv3~ to Lv1_Cmp1_Vv3
Copy angles from Lv1_Cmp1_Vv4~ to Lv1_Cmp1_Vv4
Copy angles from Lv2_Cmp1_V0~ to Lv2_Cmp1_V0
Copy angles from Lv2_Cmp1_Vh1~ to Lv2_Cmp1_Vh1
Copy angles from Lv2_Cmp1_Vh2~ to Lv2_Cmp1_Vh2
Copy angles from Lv2_Cmp1_Vh3~ to Lv2_Cmp1_Vh3
Copy angles from Lv2_Cmp1_Vh4~ to Lv2_Cmp1_Vh4
Copy angles from Lv2_Cmp1_Vv1~ to Lv2_Cmp1_Vv1
Copy angles from Lv2_Cmp1_Vv2~ to Lv2_Cmp1_Vv2
Copy angles from Lv2_Cmp1_Vv3~ to Lv2_Cmp1_Vv3
Copy angles from Lv2_Cmp1_Vv4~ to Lv2_Cmp1_Vv4
Copy angles from Lv3_Cmp1_V0~ to Lv3_Cmp1_V0
Copy angles from Lv3_Cmp1_Vh1~ to Lv3_Cmp1_Vh1
Copy angles from Lv3_Cmp1_Vh2~ to Lv3_Cmp1_Vh2
Copy angles from Lv3_Cmp1_Vh3~ to Lv3_Cmp1_Vh3
Copy angles from Lv3_Cmp1_Vh4~ to Lv3_Cmp1_Vh4
Copy angles from Lv3_Cmp1_Vv1~ to Lv3_Cmp1_Vv1
Copy angles from Lv3_Cmp1_Vv2~ to Lv3_Cmp1_Vv2
Copy angles from Lv3_Cmp1_Vv3~ to Lv3_Cmp1_Vv3
Copy angles from Lv3_Cmp1_Vv4~ to Lv3_Cmp1_Vv4
Copy angles from Lv4_Cmp1_V0~ to Lv4_Cmp1_V0
Copy angles from Lv4_Cmp1_Vh1~ to Lv4_Cmp1_Vh1
Copy angles from Lv4_Cmp1_Vh2~ to Lv4_Cmp1_Vh2
Copy angles from Lv4_Cmp1_Vh3~ to Lv4_Cmp1_Vh3
Copy angles from Lv4_Cmp1_Vh4~ to Lv4_Cmp1_Vh4
Copy angles from Lv4_Cmp1_Vv1~ to Lv4_Cmp1_Vv1
Copy angles from Lv4_Cmp1_Vv2~ to Lv4_Cmp1_Vv2
Copy angles from Lv4_Cmp1_Vv3~ to Lv4_Cmp1_Vv3
Copy angles from Lv4_Cmp1_Vv4~ to Lv4_Cmp1_Vv4

"MSE: 2.6008e-14"

単一の GPU で学習中。

エポック	反復	経過時間 (h h : mm : s s)	ミニバッチ RMSE	ミニバッチ損失	基本学
1	1	00 : 00 : 05	5.20	13.5	0.0
13	50	00 : 01 : 18	5.01	12.6	0.0
25	100	00 : 02 : 32	4.94	12.2	0.0
30	120	00 : 03 : 03	4.91	12.1	0.0

学習終了：最大エポック数が完了しました。

Copy angles from Lv1_Cmp1_V0~ to Lv1_Cmp1_V0
Copy angles from Lv1_Cmp1_Vh1~ to Lv1_Cmp1_Vh1
Copy angles from Lv1_Cmp1_Vh2~ to Lv1_Cmp1_Vh2
Copy angles from Lv1_Cmp1_Vh3~ to Lv1_Cmp1_Vh3
Copy angles from Lv1_Cmp1_Vh4~ to Lv1_Cmp1_Vh4

Copy angles from Lv1_Cmp1_Vv1~ to Lv1_Cmp1_Vv1
 Copy angles from Lv1_Cmp1_Vv2~ to Lv1_Cmp1_Vv2
 Copy angles from Lv1_Cmp1_Vv3~ to Lv1_Cmp1_Vv3
 Copy angles from Lv1_Cmp1_Vv4~ to Lv1_Cmp1_Vv4
 Copy angles from Lv2_Cmp1_V0~ to Lv2_Cmp1_V0
 Copy angles from Lv2_Cmp1_Vh1~ to Lv2_Cmp1_Vh1
 Copy angles from Lv2_Cmp1_Vh2~ to Lv2_Cmp1_Vh2
 Copy angles from Lv2_Cmp1_Vh3~ to Lv2_Cmp1_Vh3
 Copy angles from Lv2_Cmp1_Vh4~ to Lv2_Cmp1_Vh4
 Copy angles from Lv2_Cmp1_Vv1~ to Lv2_Cmp1_Vv1
 Copy angles from Lv2_Cmp1_Vv2~ to Lv2_Cmp1_Vv2
 Copy angles from Lv2_Cmp1_Vv3~ to Lv2_Cmp1_Vv3
 Copy angles from Lv2_Cmp1_Vv4~ to Lv2_Cmp1_Vv4
 Copy angles from Lv3_Cmp1_V0~ to Lv3_Cmp1_V0
 Copy angles from Lv3_Cmp1_Vh1~ to Lv3_Cmp1_Vh1
 Copy angles from Lv3_Cmp1_Vh2~ to Lv3_Cmp1_Vh2
 Copy angles from Lv3_Cmp1_Vh3~ to Lv3_Cmp1_Vh3
 Copy angles from Lv3_Cmp1_Vh4~ to Lv3_Cmp1_Vh4
 Copy angles from Lv3_Cmp1_Vv1~ to Lv3_Cmp1_Vv1
 Copy angles from Lv3_Cmp1_Vv2~ to Lv3_Cmp1_Vv2
 Copy angles from Lv3_Cmp1_Vv3~ to Lv3_Cmp1_Vv3
 Copy angles from Lv3_Cmp1_Vv4~ to Lv3_Cmp1_Vv4
 Copy angles from Lv4_Cmp1_V0~ to Lv4_Cmp1_V0
 Copy angles from Lv4_Cmp1_Vh1~ to Lv4_Cmp1_Vh1
 Copy angles from Lv4_Cmp1_Vh2~ to Lv4_Cmp1_Vh2
 Copy angles from Lv4_Cmp1_Vh3~ to Lv4_Cmp1_Vh3
 Copy angles from Lv4_Cmp1_Vh4~ to Lv4_Cmp1_Vh4
 Copy angles from Lv4_Cmp1_Vv1~ to Lv4_Cmp1_Vv1
 Copy angles from Lv4_Cmp1_Vv2~ to Lv4_Cmp1_Vv2
 Copy angles from Lv4_Cmp1_Vv3~ to Lv4_Cmp1_Vv3
 Copy angles from Lv4_Cmp1_Vv4~ to Lv4_Cmp1_Vv4

"MSE: 2.5606e-14"

単一の GPU で学習中。

エポック	反復	経過時間 (h h : mm : s s)	ミニバッチ RMSE	ミニバッチ損失	基本学
1	1	00 : 00 : 04	4.92	12.1	0.0
13	50	00 : 01 : 18	4.96	12.3	0.0
25	100	00 : 02 : 32	4.92	12.1	0.0
30	120	00 : 03 : 03	5.00	12.5	0.0

学習終了: 最大エポック数が完了しました。

Copy angles from Lv1_Cmp1_V0~ to Lv1_Cmp1_V0
 Copy angles from Lv1_Cmp1_Vh1~ to Lv1_Cmp1_Vh1
 Copy angles from Lv1_Cmp1_Vh2~ to Lv1_Cmp1_Vh2
 Copy angles from Lv1_Cmp1_Vh3~ to Lv1_Cmp1_Vh3
 Copy angles from Lv1_Cmp1_Vh4~ to Lv1_Cmp1_Vh4
 Copy angles from Lv1_Cmp1_Vv1~ to Lv1_Cmp1_Vv1
 Copy angles from Lv1_Cmp1_Vv2~ to Lv1_Cmp1_Vv2
 Copy angles from Lv1_Cmp1_Vv3~ to Lv1_Cmp1_Vv3
 Copy angles from Lv1_Cmp1_Vv4~ to Lv1_Cmp1_Vv4
 Copy angles from Lv2_Cmp1_V0~ to Lv2_Cmp1_V0
 Copy angles from Lv2_Cmp1_Vh1~ to Lv2_Cmp1_Vh1
 Copy angles from Lv2_Cmp1_Vh2~ to Lv2_Cmp1_Vh2
 Copy angles from Lv2_Cmp1_Vh3~ to Lv2_Cmp1_Vh3
 Copy angles from Lv2_Cmp1_Vh4~ to Lv2_Cmp1_Vh4
 Copy angles from Lv2_Cmp1_Vv1~ to Lv2_Cmp1_Vv1
 Copy angles from Lv2_Cmp1_Vv2~ to Lv2_Cmp1_Vv2
 Copy angles from Lv2_Cmp1_Vv3~ to Lv2_Cmp1_Vv3
 Copy angles from Lv2_Cmp1_Vv4~ to Lv2_Cmp1_Vv4
 Copy angles from Lv3_Cmp1_V0~ to Lv3_Cmp1_V0
 Copy angles from Lv3_Cmp1_Vh1~ to Lv3_Cmp1_Vh1
 Copy angles from Lv3_Cmp1_Vh2~ to Lv3_Cmp1_Vh2

Copy angles from Lv3_Cmp1_Vh3~ to Lv3_Cmp1_Vh3
 Copy angles from Lv3_Cmp1_Vh4~ to Lv3_Cmp1_Vh4
 Copy angles from Lv3_Cmp1_Vv1~ to Lv3_Cmp1_Vv1
 Copy angles from Lv3_Cmp1_Vv2~ to Lv3_Cmp1_Vv2
 Copy angles from Lv3_Cmp1_Vv3~ to Lv3_Cmp1_Vv3
 Copy angles from Lv3_Cmp1_Vv4~ to Lv3_Cmp1_Vv4
 Copy angles from Lv4_Cmp1_V0~ to Lv4_Cmp1_V0
 Copy angles from Lv4_Cmp1_Vh1~ to Lv4_Cmp1_Vh1
 Copy angles from Lv4_Cmp1_Vh2~ to Lv4_Cmp1_Vh2
 Copy angles from Lv4_Cmp1_Vh3~ to Lv4_Cmp1_Vh3
 Copy angles from Lv4_Cmp1_Vh4~ to Lv4_Cmp1_Vh4
 Copy angles from Lv4_Cmp1_Vv1~ to Lv4_Cmp1_Vv1
 Copy angles from Lv4_Cmp1_Vv2~ to Lv4_Cmp1_Vv2
 Copy angles from Lv4_Cmp1_Vv3~ to Lv4_Cmp1_Vv3
 Copy angles from Lv4_Cmp1_Vv4~ to Lv4_Cmp1_Vv4

"MSE: 2.7025e-14"

単一の GPU で学習中。

エポック	反復	経過時間 (h h : mm : s s)	ミニバッチ RMSE	ミニバッチ損失	基本学
1	1	00 : 00 : 04	5.16	13.3	0.0
13	50	00 : 01 : 18	4.94	12.2	0.0
25	100	00 : 02 : 33	4.98	12.4	0.0
30	120	00 : 03 : 04	5.11	13.0	0.0

学習終了: 最大エポック数が完了しました。

Copy angles from Lv1_Cmp1_V0~ to Lv1_Cmp1_V0
 Copy angles from Lv1_Cmp1_Vh1~ to Lv1_Cmp1_Vh1
 Copy angles from Lv1_Cmp1_Vh2~ to Lv1_Cmp1_Vh2
 Copy angles from Lv1_Cmp1_Vh3~ to Lv1_Cmp1_Vh3
 Copy angles from Lv1_Cmp1_Vh4~ to Lv1_Cmp1_Vh4
 Copy angles from Lv1_Cmp1_Vv1~ to Lv1_Cmp1_Vv1
 Copy angles from Lv1_Cmp1_Vv2~ to Lv1_Cmp1_Vv2
 Copy angles from Lv1_Cmp1_Vv3~ to Lv1_Cmp1_Vv3
 Copy angles from Lv1_Cmp1_Vv4~ to Lv1_Cmp1_Vv4
 Copy angles from Lv2_Cmp1_V0~ to Lv2_Cmp1_V0
 Copy angles from Lv2_Cmp1_Vh1~ to Lv2_Cmp1_Vh1
 Copy angles from Lv2_Cmp1_Vh2~ to Lv2_Cmp1_Vh2
 Copy angles from Lv2_Cmp1_Vh3~ to Lv2_Cmp1_Vh3
 Copy angles from Lv2_Cmp1_Vh4~ to Lv2_Cmp1_Vh4
 Copy angles from Lv2_Cmp1_Vv1~ to Lv2_Cmp1_Vv1
 Copy angles from Lv2_Cmp1_Vv2~ to Lv2_Cmp1_Vv2
 Copy angles from Lv2_Cmp1_Vv3~ to Lv2_Cmp1_Vv3
 Copy angles from Lv2_Cmp1_Vv4~ to Lv2_Cmp1_Vv4
 Copy angles from Lv3_Cmp1_V0~ to Lv3_Cmp1_V0
 Copy angles from Lv3_Cmp1_Vh1~ to Lv3_Cmp1_Vh1
 Copy angles from Lv3_Cmp1_Vh2~ to Lv3_Cmp1_Vh2
 Copy angles from Lv3_Cmp1_Vh3~ to Lv3_Cmp1_Vh3
 Copy angles from Lv3_Cmp1_Vh4~ to Lv3_Cmp1_Vh4
 Copy angles from Lv3_Cmp1_Vv1~ to Lv3_Cmp1_Vv1
 Copy angles from Lv3_Cmp1_Vv2~ to Lv3_Cmp1_Vv2
 Copy angles from Lv3_Cmp1_Vv3~ to Lv3_Cmp1_Vv3
 Copy angles from Lv3_Cmp1_Vv4~ to Lv3_Cmp1_Vv4
 Copy angles from Lv4_Cmp1_V0~ to Lv4_Cmp1_V0
 Copy angles from Lv4_Cmp1_Vh1~ to Lv4_Cmp1_Vh1
 Copy angles from Lv4_Cmp1_Vh2~ to Lv4_Cmp1_Vh2
 Copy angles from Lv4_Cmp1_Vh3~ to Lv4_Cmp1_Vh3
 Copy angles from Lv4_Cmp1_Vh4~ to Lv4_Cmp1_Vh4
 Copy angles from Lv4_Cmp1_Vv1~ to Lv4_Cmp1_Vv1
 Copy angles from Lv4_Cmp1_Vv2~ to Lv4_Cmp1_Vv2
 Copy angles from Lv4_Cmp1_Vv3~ to Lv4_Cmp1_Vv3
 Copy angles from Lv4_Cmp1_Vv4~ to Lv4_Cmp1_Vv4

"MSE: 3.023e-14"

単一の GPU で学習中。

エポック	反復	経過時間 (h h : mm : s s)	ミニバッチ RMSE	ミニバッチ損失	基本学
1	1	00 : 00 : 04	4.93	12.1	0.0
13	50	00 : 01 : 18	5.08	12.9	0.0
25	100	00 : 02 : 32	5.13	13.2	0.0
30	120	00 : 03 : 03	4.87	11.9	0.0

学習終了：最大エポック数が完了しました。

Copy angles from Lv1_Cmp1_V0~ to Lv1_Cmp1_V0
Copy angles from Lv1_Cmp1_Vh1~ to Lv1_Cmp1_Vh1
Copy angles from Lv1_Cmp1_Vh2~ to Lv1_Cmp1_Vh2
Copy angles from Lv1_Cmp1_Vh3~ to Lv1_Cmp1_Vh3
Copy angles from Lv1_Cmp1_Vh4~ to Lv1_Cmp1_Vh4
Copy angles from Lv1_Cmp1_Vv1~ to Lv1_Cmp1_Vv1
Copy angles from Lv1_Cmp1_Vv2~ to Lv1_Cmp1_Vv2
Copy angles from Lv1_Cmp1_Vv3~ to Lv1_Cmp1_Vv3
Copy angles from Lv1_Cmp1_Vv4~ to Lv1_Cmp1_Vv4
Copy angles from Lv2_Cmp1_V0~ to Lv2_Cmp1_V0
Copy angles from Lv2_Cmp1_Vh1~ to Lv2_Cmp1_Vh1
Copy angles from Lv2_Cmp1_Vh2~ to Lv2_Cmp1_Vh2
Copy angles from Lv2_Cmp1_Vh3~ to Lv2_Cmp1_Vh3
Copy angles from Lv2_Cmp1_Vh4~ to Lv2_Cmp1_Vh4
Copy angles from Lv2_Cmp1_Vv1~ to Lv2_Cmp1_Vv1
Copy angles from Lv2_Cmp1_Vv2~ to Lv2_Cmp1_Vv2
Copy angles from Lv2_Cmp1_Vv3~ to Lv2_Cmp1_Vv3
Copy angles from Lv2_Cmp1_Vv4~ to Lv2_Cmp1_Vv4
Copy angles from Lv3_Cmp1_V0~ to Lv3_Cmp1_V0
Copy angles from Lv3_Cmp1_Vh1~ to Lv3_Cmp1_Vh1
Copy angles from Lv3_Cmp1_Vh2~ to Lv3_Cmp1_Vh2
Copy angles from Lv3_Cmp1_Vh3~ to Lv3_Cmp1_Vh3
Copy angles from Lv3_Cmp1_Vh4~ to Lv3_Cmp1_Vh4
Copy angles from Lv3_Cmp1_Vv1~ to Lv3_Cmp1_Vv1
Copy angles from Lv3_Cmp1_Vv2~ to Lv3_Cmp1_Vv2
Copy angles from Lv3_Cmp1_Vv3~ to Lv3_Cmp1_Vv3
Copy angles from Lv3_Cmp1_Vv4~ to Lv3_Cmp1_Vv4
Copy angles from Lv4_Cmp1_V0~ to Lv4_Cmp1_V0
Copy angles from Lv4_Cmp1_Vh1~ to Lv4_Cmp1_Vh1
Copy angles from Lv4_Cmp1_Vh2~ to Lv4_Cmp1_Vh2
Copy angles from Lv4_Cmp1_Vh3~ to Lv4_Cmp1_Vh3
Copy angles from Lv4_Cmp1_Vh4~ to Lv4_Cmp1_Vh4
Copy angles from Lv4_Cmp1_Vv1~ to Lv4_Cmp1_Vv1
Copy angles from Lv4_Cmp1_Vv2~ to Lv4_Cmp1_Vv2
Copy angles from Lv4_Cmp1_Vv3~ to Lv4_Cmp1_Vv3
Copy angles from Lv4_Cmp1_Vv4~ to Lv4_Cmp1_Vv4

"MSE: 2.4277e-14"

単一の GPU で学習中。

エポック	反復	経過時間 (h h : mm : s s)	ミニバッチ RMSE	ミニバッチ損失	基本学
1	1	00 : 00 : 04	5.13	13.2	0.0
13	50	00 : 01 : 19	5.24	13.7	0.0
25	100	00 : 02 : 34	5.03	12.6	0.0
30	120	00 : 03 : 05	5.11	13.1	0.0

学習終了：最大エポック数が完了しました。

Copy angles from Lv1_Cmp1_V0~ to Lv1_Cmp1_V0
Copy angles from Lv1_Cmp1_Vh1~ to Lv1_Cmp1_Vh1
Copy angles from Lv1_Cmp1_Vh2~ to Lv1_Cmp1_Vh2
Copy angles from Lv1_Cmp1_Vh3~ to Lv1_Cmp1_Vh3
Copy angles from Lv1_Cmp1_Vh4~ to Lv1_Cmp1_Vh4

Copy angles from Lv1_Cmp1_Vv1~ to Lv1_Cmp1_Vv1
 Copy angles from Lv1_Cmp1_Vv2~ to Lv1_Cmp1_Vv2
 Copy angles from Lv1_Cmp1_Vv3~ to Lv1_Cmp1_Vv3
 Copy angles from Lv1_Cmp1_Vv4~ to Lv1_Cmp1_Vv4
 Copy angles from Lv2_Cmp1_V0~ to Lv2_Cmp1_V0
 Copy angles from Lv2_Cmp1_Vh1~ to Lv2_Cmp1_Vh1
 Copy angles from Lv2_Cmp1_Vh2~ to Lv2_Cmp1_Vh2
 Copy angles from Lv2_Cmp1_Vh3~ to Lv2_Cmp1_Vh3
 Copy angles from Lv2_Cmp1_Vh4~ to Lv2_Cmp1_Vh4
 Copy angles from Lv2_Cmp1_Vv1~ to Lv2_Cmp1_Vv1
 Copy angles from Lv2_Cmp1_Vv2~ to Lv2_Cmp1_Vv2
 Copy angles from Lv2_Cmp1_Vv3~ to Lv2_Cmp1_Vv3
 Copy angles from Lv2_Cmp1_Vv4~ to Lv2_Cmp1_Vv4
 Copy angles from Lv3_Cmp1_V0~ to Lv3_Cmp1_V0
 Copy angles from Lv3_Cmp1_Vh1~ to Lv3_Cmp1_Vh1
 Copy angles from Lv3_Cmp1_Vh2~ to Lv3_Cmp1_Vh2
 Copy angles from Lv3_Cmp1_Vh3~ to Lv3_Cmp1_Vh3
 Copy angles from Lv3_Cmp1_Vh4~ to Lv3_Cmp1_Vh4
 Copy angles from Lv3_Cmp1_Vv1~ to Lv3_Cmp1_Vv1
 Copy angles from Lv3_Cmp1_Vv2~ to Lv3_Cmp1_Vv2
 Copy angles from Lv3_Cmp1_Vv3~ to Lv3_Cmp1_Vv3
 Copy angles from Lv3_Cmp1_Vv4~ to Lv3_Cmp1_Vv4
 Copy angles from Lv4_Cmp1_V0~ to Lv4_Cmp1_V0
 Copy angles from Lv4_Cmp1_Vh1~ to Lv4_Cmp1_Vh1
 Copy angles from Lv4_Cmp1_Vh2~ to Lv4_Cmp1_Vh2
 Copy angles from Lv4_Cmp1_Vh3~ to Lv4_Cmp1_Vh3
 Copy angles from Lv4_Cmp1_Vh4~ to Lv4_Cmp1_Vh4
 Copy angles from Lv4_Cmp1_Vv1~ to Lv4_Cmp1_Vv1
 Copy angles from Lv4_Cmp1_Vv2~ to Lv4_Cmp1_Vv2
 Copy angles from Lv4_Cmp1_Vv3~ to Lv4_Cmp1_Vv3
 Copy angles from Lv4_Cmp1_Vv4~ to Lv4_Cmp1_Vv4

"MSE: 2.5847e-14"

単一の GPU で学習中。

エポック	反復	経過時間 (h h : mm : s s)	ミニバッチ RMSE	ミニバッチ損失	基本学
1	1	00 : 00 : 04	5.19	13.5	0.0
13	50	00 : 01 : 18	5.16	13.3	0.0
25	100	00 : 02 : 32	5.11	13.1	0.0
30	120	00 : 03 : 03	4.98	12.4	0.0

学習終了: 最大エポック数が完了しました。

Copy angles from Lv1_Cmp1_V0~ to Lv1_Cmp1_V0
 Copy angles from Lv1_Cmp1_Vh1~ to Lv1_Cmp1_Vh1
 Copy angles from Lv1_Cmp1_Vh2~ to Lv1_Cmp1_Vh2
 Copy angles from Lv1_Cmp1_Vh3~ to Lv1_Cmp1_Vh3
 Copy angles from Lv1_Cmp1_Vh4~ to Lv1_Cmp1_Vh4
 Copy angles from Lv1_Cmp1_Vv1~ to Lv1_Cmp1_Vv1
 Copy angles from Lv1_Cmp1_Vv2~ to Lv1_Cmp1_Vv2
 Copy angles from Lv1_Cmp1_Vv3~ to Lv1_Cmp1_Vv3
 Copy angles from Lv1_Cmp1_Vv4~ to Lv1_Cmp1_Vv4
 Copy angles from Lv2_Cmp1_V0~ to Lv2_Cmp1_V0
 Copy angles from Lv2_Cmp1_Vh1~ to Lv2_Cmp1_Vh1
 Copy angles from Lv2_Cmp1_Vh2~ to Lv2_Cmp1_Vh2
 Copy angles from Lv2_Cmp1_Vh3~ to Lv2_Cmp1_Vh3
 Copy angles from Lv2_Cmp1_Vh4~ to Lv2_Cmp1_Vh4
 Copy angles from Lv2_Cmp1_Vv1~ to Lv2_Cmp1_Vv1
 Copy angles from Lv2_Cmp1_Vv2~ to Lv2_Cmp1_Vv2
 Copy angles from Lv2_Cmp1_Vv3~ to Lv2_Cmp1_Vv3
 Copy angles from Lv2_Cmp1_Vv4~ to Lv2_Cmp1_Vv4
 Copy angles from Lv3_Cmp1_V0~ to Lv3_Cmp1_V0
 Copy angles from Lv3_Cmp1_Vh1~ to Lv3_Cmp1_Vh1
 Copy angles from Lv3_Cmp1_Vh2~ to Lv3_Cmp1_Vh2


```

Copy angles from Lv3_Cmp1_Vh3~ to Lv3_Cmp1_Vh3
Copy angles from Lv3_Cmp1_Vh4~ to Lv3_Cmp1_Vh4
Copy angles from Lv3_Cmp1_Vv1~ to Lv3_Cmp1_Vv1
Copy angles from Lv3_Cmp1_Vv2~ to Lv3_Cmp1_Vv2
Copy angles from Lv3_Cmp1_Vv3~ to Lv3_Cmp1_Vv3
Copy angles from Lv3_Cmp1_Vv4~ to Lv3_Cmp1_Vv4
Copy angles from Lv4_Cmp1_V0~ to Lv4_Cmp1_V0
Copy angles from Lv4_Cmp1_Vh1~ to Lv4_Cmp1_Vh1
Copy angles from Lv4_Cmp1_Vh2~ to Lv4_Cmp1_Vh2
Copy angles from Lv4_Cmp1_Vh3~ to Lv4_Cmp1_Vh3
Copy angles from Lv4_Cmp1_Vh4~ to Lv4_Cmp1_Vh4
Copy angles from Lv4_Cmp1_Vv1~ to Lv4_Cmp1_Vv1
Copy angles from Lv4_Cmp1_Vv2~ to Lv4_Cmp1_Vv2
Copy angles from Lv4_Cmp1_Vv3~ to Lv4_Cmp1_Vv3
Copy angles from Lv4_Cmp1_Vv4~ to Lv4_Cmp1_Vv4
"MSE: 3.259e-14"
Copy angles from Lv1_Cmp1_V0~ to Lv1_Cmp1_V0
Copy angles from Lv1_Cmp1_Vh1~ to Lv1_Cmp1_Vh1
Copy angles from Lv1_Cmp1_Vh2~ to Lv1_Cmp1_Vh2
Copy angles from Lv1_Cmp1_Vh3~ to Lv1_Cmp1_Vh3
Copy angles from Lv1_Cmp1_Vh4~ to Lv1_Cmp1_Vh4
Copy angles from Lv1_Cmp1_Vv1~ to Lv1_Cmp1_Vv1
Copy angles from Lv1_Cmp1_Vv2~ to Lv1_Cmp1_Vv2
Copy angles from Lv1_Cmp1_Vv3~ to Lv1_Cmp1_Vv3
Copy angles from Lv1_Cmp1_Vv4~ to Lv1_Cmp1_Vv4
Copy angles from Lv2_Cmp1_V0~ to Lv2_Cmp1_V0
Copy angles from Lv2_Cmp1_Vh1~ to Lv2_Cmp1_Vh1
Copy angles from Lv2_Cmp1_Vh2~ to Lv2_Cmp1_Vh2
Copy angles from Lv2_Cmp1_Vh3~ to Lv2_Cmp1_Vh3
Copy angles from Lv2_Cmp1_Vh4~ to Lv2_Cmp1_Vh4
Copy angles from Lv2_Cmp1_Vv1~ to Lv2_Cmp1_Vv1
Copy angles from Lv2_Cmp1_Vv2~ to Lv2_Cmp1_Vv2
Copy angles from Lv2_Cmp1_Vv3~ to Lv2_Cmp1_Vv3
Copy angles from Lv2_Cmp1_Vv4~ to Lv2_Cmp1_Vv4
Copy angles from Lv3_Cmp1_V0~ to Lv3_Cmp1_V0
Copy angles from Lv3_Cmp1_Vh1~ to Lv3_Cmp1_Vh1
Copy angles from Lv3_Cmp1_Vh2~ to Lv3_Cmp1_Vh2
Copy angles from Lv3_Cmp1_Vh3~ to Lv3_Cmp1_Vh3
Copy angles from Lv3_Cmp1_Vh4~ to Lv3_Cmp1_Vh4
Copy angles from Lv3_Cmp1_Vv1~ to Lv3_Cmp1_Vv1
Copy angles from Lv3_Cmp1_Vv2~ to Lv3_Cmp1_Vv2
Copy angles from Lv3_Cmp1_Vv3~ to Lv3_Cmp1_Vv3
Copy angles from Lv3_Cmp1_Vv4~ to Lv3_Cmp1_Vv4
Copy angles from Lv4_Cmp1_V0~ to Lv4_Cmp1_V0
Copy angles from Lv4_Cmp1_Vh1~ to Lv4_Cmp1_Vh1
Copy angles from Lv4_Cmp1_Vh2~ to Lv4_Cmp1_Vh2
Copy angles from Lv4_Cmp1_Vh3~ to Lv4_Cmp1_Vh3
Copy angles from Lv4_Cmp1_Vh4~ to Lv4_Cmp1_Vh4
Copy angles from Lv4_Cmp1_Vv1~ to Lv4_Cmp1_Vv1
Copy angles from Lv4_Cmp1_Vv2~ to Lv4_Cmp1_Vv2
Copy angles from Lv4_Cmp1_Vv3~ to Lv4_Cmp1_Vv3
Copy angles from Lv4_Cmp1_Vv4~ to Lv4_Cmp1_Vv4

```

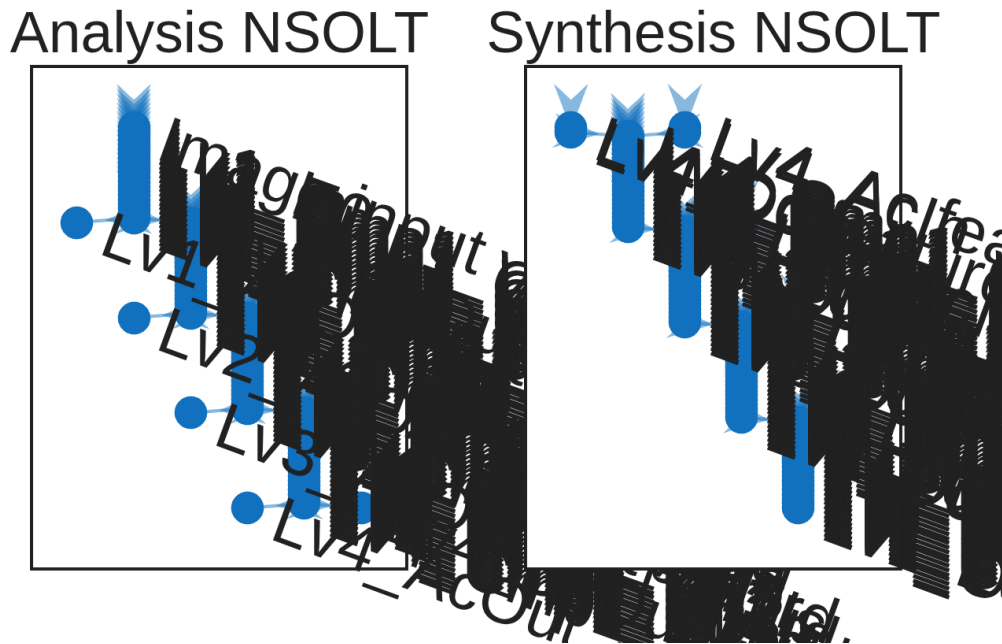
```

analysislgraph = layerGraph(analysisnet);
synthesislgraph = layerGraph(synthesisnet);

figure
subplot(1,2,1)
plot(analysislgraph)
title('Analysis NSOLT')
subplot(1,2,2)

```

```
plot(synthesislgraph)
title('Synthesis NSOLT')
```



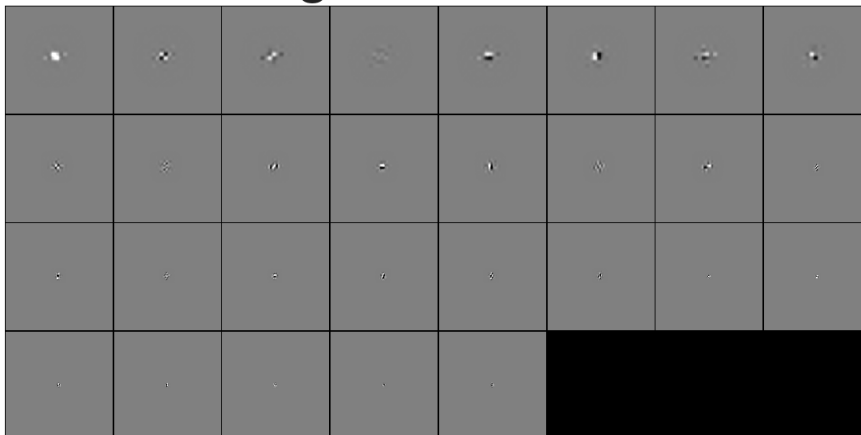
要素画像の表示

```
import saivdr.dcn.*

figure

atomicimshow(synthesisnet, [], 2^(nLevels-1))
title('Atomic images of trained NSOLT')
```

Atomic images of trained NSOLT



推論用 **NSOLT** ネットワークの構築

dlnetwork は（DAGNetwork と異なり）入力層のサイズに縛られず任意の画像サイズで順伝播できるため、学習時の analysisnet/synthesisnet をそのまま推論に用いる。これにより assembleNetwork 用の層差し替えが不要になり、predict 呼び出しも約 1.6 倍高速化する。

随伴関係（完全再構成）の確認

NSOLT はパーセバルタイト性を満たす。

```
u = rand(szOrg, 'single');
dlu = dlarray(u, 'SSCB');
[s{1:nLevels+1}] = analysisnet.predict(dlu);
dlv = synthesisnet.predict(s{1:nLevels+1});
assert(mse(extractdata(dlv),u)<1e-9)
```

NSOLT による合成処理とその随伴処理の定義

レベルごとに単一段（1 レベル分）の要素画像（等価 FIR フィルタ）を学習済みネットワークから抽出し、レベルの深さ分だけ dlconv/dltranspconv を再帰的に適用する階層的（カスケード）方式で高速化する。全レベルの係数を単一の畳み込みに平坦化する方式は各レベルの解像度差（decFactor^{level}）を無視してしまい多レベルでは完全再構成を満たせないため、必ずレベルごとに畳み込みを重ねる。

```
import saivdr.dcnnc.*
maxDecFactor1 = decFactor; % 単一レベル分の等価フィルタ長を計算 (nLevels=1 相当)
szFilters1 = maxDecFactor1 + ppOrder.*decFactor.*(maxDecFactor1-1)./
(decFactor-1);
protolgraph = fcn_creatensoltlgraph2d([], ...
    'InputSize',szFilters1,...
    'NumberOfChannels',nChannels,...
    'DecimationFactor',decFactor,...
    'PolyPhaseOrder',ppOrder,...
    'NumberOfLevels',1,...
    'NumberOfVanishingMoments',noDcLeakage,...
    'Mode','Synthesizer');
protonetTemplate = dlnetwork(protolgraph);
nsoltconfig.nLevels = nLevels;
nsoltconfig.nChsPerLv = sum(nChannels);
nsoltconfig.decFactor = decFactor;
nsoltconfig.szOrg = szOrg;
nsoltconfig.padSz = (szFilters1 - decFactor)/2;
nsoltconfig.W = cell(1,nLevels);
for iLv = 1:nLevels
    protoLv = cplevelangles(synthesisnet,protonetTemplate,iLv);
    nsoltconfig.W{iLv} = single(getatomicimages(protoLv,szFilters1,1));
end
syn_nsolt = @(x) synthesisnsolt(x,nsoltconfig);
adj_nsolt = @(y) analysisnsolt(y,nsoltconfig);
```

随伴関係の確認

```

x = adj_nsolt(y);
v = randn(size(x));
u = syn_nsolt(v);
assert(abs(dot(y(:),u(:))-dot(x(:),v(:)))<1e-3)

```

繰返しハード閾値処理(IHT)によるスパース近似の比較

辞書の準備

```

blkdctwon = { syn_blkdict, adj_blkdict, "Block DCTwon", false };
blkdict = { syn_blkdict, adj_blkdict, "Block DCT", true };
blkpcawon = { syn_blkpca, adj_blkpca, "Block PCAwon", false };
blkpca = { syn_blkpca, adj_blkpca, "Block PCA", true };
blkkrca = { syn_blkkrca, adj_blkkrca, "Block RICA", true };
blkksvd = { syn_blkksvd, adj_blkksvd, "Block K-SVD", true };
nsoltwon = { syn_nsolt, adj_nsolt, "NSOLTwon", false };
nsolt = { syn_nsolt, adj_nsolt, "NSOLT", true };
dicset = { blkdctwon, blkdict, blkpcawon, blkpca, blkkrca, blkksvd,
nsoltwon, nsolt };
nDics = length(dicset);

```

IHT

$$\mathbf{x}^{(t+1)} \leftarrow \mathcal{H}_{BK}(\mathbf{x}^{(t)} + \mu^{(t)} \hat{\mathbf{D}}^T (\mathbf{y} - \hat{\mathbf{D}} \mathbf{x}^{(t)}))$$

$t \leftarrow t + 1$

- T. Blumensath and M. E. Davies, "Normalized Iterative Hard Thresholding: Guaranteed Stability and Performance," in IEEE Journal of Selected Topics in Signal Processing, vol. 4, no. 2, pp. 298-309, April 2010, doi: 10.1109/JSTSP.2010.2042411.

```

nItersIht = 2000;

% 平均値を引いた画像を用意（近似後に平均値を加算）
ymean = mean(yorg, "all");
y = yorg - ymean;
% 準備
c = 1e-3;
kappa = 1.1/(1-c);
nCoefs = floor(sparsityRatio*prod(szOrg));
psnrs = zeros(nItersIht, nDics);
ssims = zeros(nItersIht, nDics);
yaprxs = cell(1, nDics);
% 繰返し処理
for iDic = 1:nDics
    dic_ = dicset{iDic};
    synproc = dic_{1};
    adjproc = dic_{2};
    dicname = dic_{3};
    isStepSizeNormalized = dic_{4};

```

```

% IHT
display(dicname)
s = adjproc(y); %  $D^T y$ 
xt = zeros(size(s), 'like', s); %  $x_1 = 0$ ;
yaprx_ = zeros(size(y), 'like', y); %  $\text{synproc}(x_1)=0$  に相当。次段の  $gt$  計算で
% 前反復の  $\text{synproc}(xt)$  結果を再利用し、毎反復 2 回計算していた  $\text{synproc}$  の呼出しを 1 回
に減らす
if isStepSizeNormalized % 正規化あり
    suppt = find(hardthresh(s, nCoefs)); %  $\Gamma_1 = \text{supp}(H_K(D^T y))$ 
    maskt = (abs(s)~=0);
end
for iIter=1:nItersIht
    % Gradient descent
    gt = adjproc(y-yaprx_); %  $g = D^T(y - Dx_n)$ 
    if ~isStepSizeNormalized % 正規化なし
        mu = (1-c);
        xtp1 = hardthresh(xt+mu*gt, nCoefs);
    else % 正規化あり
        ggt = gt(suppt); %  $g_{\Gamma_n}$ 
        ugt = synproc(maskt.*gt); %  $D_{\Gamma_n}^T g_{\Gamma_n}$ 
        mu = (ggt.'*ggt)/(ugt(:).'*ugt(:));
        ttp1 = hardthresh(xt+mu*gt, nCoefs); %  $\tilde{x}_{n+1} = H_K(x_n + \mu_n g_n)$ 
        supptp1 = find(ttp1); %  $\Gamma_{n+1} = \text{supp}(\tilde{x}_{n+1})$ 
        if length(supptp1)==length(suppt) && all(supptp1==suppt)
            xtp1 = ttp1; %  $x_{n+1} = \tilde{x}_{n+1}$ 
        else
            dxt = ttp1-xt; %  $\tilde{x}_{n+1} - x_n$ 
            omega = (1-c)*(norm(dxt, 'fro')/norm(synproc(dxt), 'fro'))^2;
            if mu <= omega
                xtp1 = ttp1; %  $x_{n+1} = \tilde{x}_{n+1}$ 
            else
                while mu > omega
                    mu = mu/(kappa*(1-c));
                    ttp1 = hardthresh(xt+mu*gt, nCoefs); %  $\tilde{x}_{n+1} = H_K(x_n$ 
+  $\mu_n g_n)$ 
                    dxt = ttp1-xt; %  $\tilde{x}_{n+1} - x_n$ 
                    omega = (1-c)*(norm(dxt, 'fro')/
norm(synproc(dxt), 'fro'))^2;
                end
                supptp1 = find(ttp1); %  $\Gamma_{n+1} = \text{supp}(\tilde{x}_{n+1})$ 
                xtp1 = ttp1; %  $x_{n+1} = \tilde{x}_{n+1}$ 
            end
        end
    end
    % Update
    suppt = supptp1;
    maskt = zeros(size(maskt), 'like', maskt);
    maskt(suppt) = 1;
end
xt = xtp1;
% Monitoring

```

```

        checkSparsity = nnz(xt)/prod(szOrg)<=sparsityRatio;
        assert(checkSparsity)
        yaprx_ = synproc(xt); % 次反復の gt 計算でも再利用するのでキャッシュしておく
        psnr_ = psnr(cast(yaprx_, 'like', y), y);
        ssim_ = ssim(cast(yaprx_, 'like', y), y);
        psnrs(iIter, iDic) = psnr_;
        ssims(iIter, iDic) = ssim_;
        fprintf("IHT(%d) PSNR: %6.4f\n", iIter, psnr_);
    end
    yaprxs{iDic} = yaprx_ + ymean;
end

```

```

dicname =
"Block DCTwoN"
dicname =
"Block DCT"
dicname =
"Block PCAwoN"
dicname =
"Block PCA"
dicname =
"Block RICA"
dicname =
"Block K-SVD"
dicname =
"NSOLTwoN"
dicname =
"NSOLT"

```

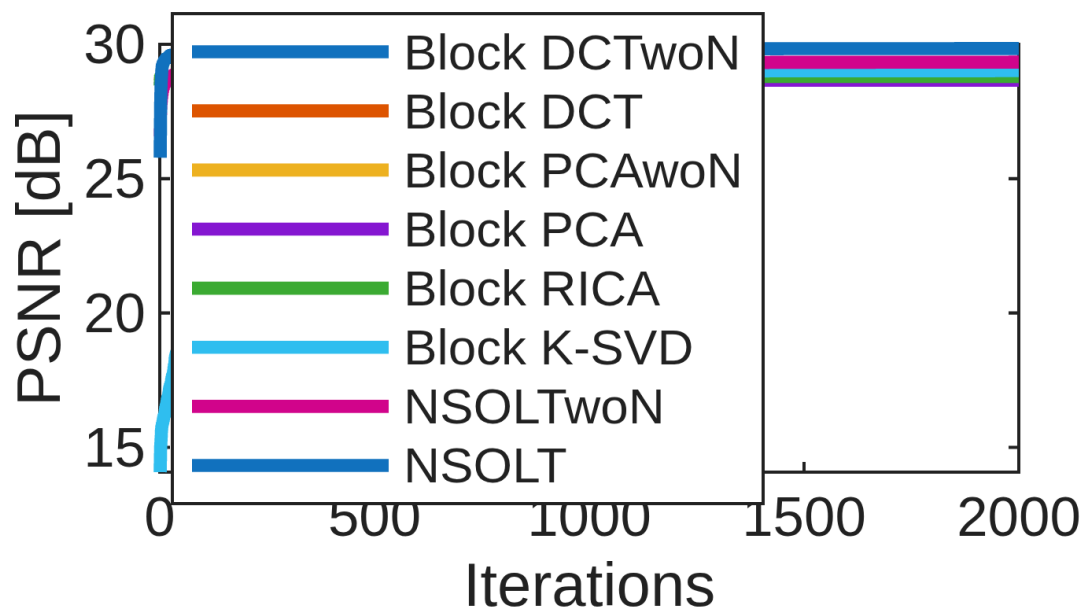
近似結果の表示

```

dicnames =
[blkdctwon{3}, blkdct{3}, blkpcawon{3}, blkpca{3}, blkrica{3}, blkksvd{3}, nsoltwo
n{3}, nsolt{3}];
psnrtbl = array2table(psnrs, 'VariableNames', dicnames);
psnrtbl =
horzcat(table((1:nItrsIht).', 'VariableNames', "Iterations"), psnrtbl);
ssimtbl = array2table(ssims, 'VariableNames', dicnames);
ssimtbl =
horzcat(table((1:nItrsIht).', 'VariableNames', "Iterations"), ssimtbl);

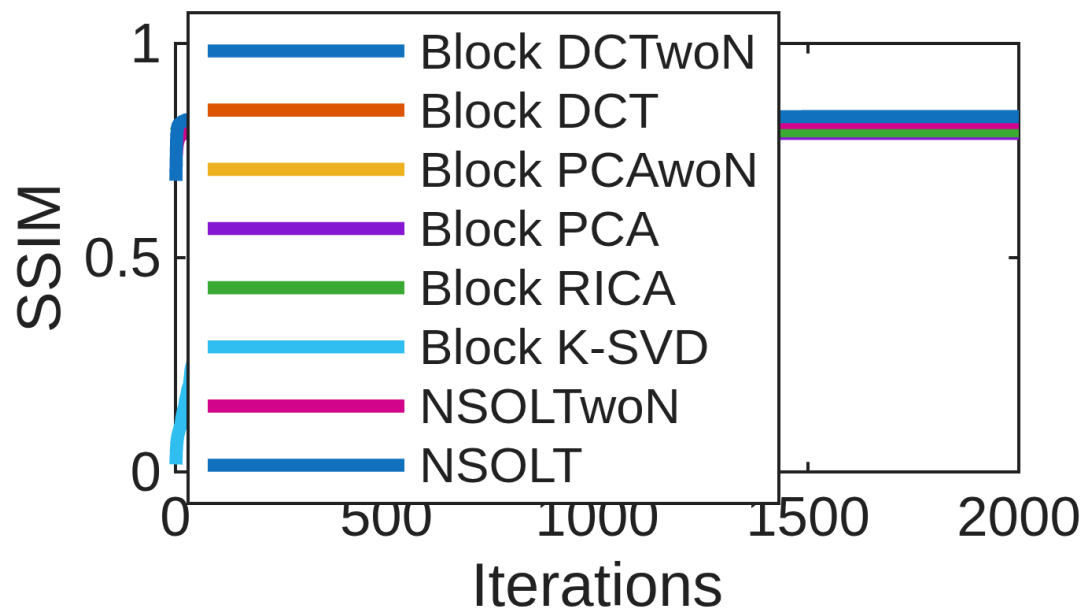
% PSNR のグラフ
figure
plot(psnrtbl, "Iterations", dicnames, 'LineWidth', 2)
ylabel('PSNR [dB]')
legend('Location', 'best')

```



% SSIM のグラフ

```
figure
plot(ssimtbl, "Iterations", dicnames, 'LineWidth', 2)
ylabel('SSIM')
legend('Location', 'best')
```



% 原画像の表示

```
figure
```

```

tiledlayout(2,ceil((nDics+1)/2))
nexttile
imshow(yorg)
title("Original image")
% 近似画像の表示
for idx = 1:nDics
    yaprx = yaprxs{idx};
    dicname = dicnames(idx)
    file_yaprx = "../results/yaprx_" + replace(lower(dicname), ' ', '_')
    + ".png";
    imwrite(yaprx,file_yaprx)
    %
    nexttile
    imshow(yaprxs{idx})
    title(dicname+" "+num2str(psnrs(end,idx))+" dB")
end

```

```

dicname =
"Block DCTwoN"
dicname =
"Block DCT"
dicname =
"Block PCAwoN"
dicname =
"Block PCA"
dicname =
"Block RICA"
dicname =
"Block K-SVD"
dicname =
"NSOLTwoN"
dicname =
"NSOLT"

```

Original Image Block DCTwoN Block DCT Block PCAwoN Block PCA Block RICA Block K-SVD NSOLTwoN NSOLT

Block DCTwoN 28.85 dB Block DCT 28.85 dB Block PCAwoN 28.85 dB Block PCA 28.85 dB Block RICA 28.85 dB Block K-SVD 28.85 dB NSOLTwoN 28.85 dB NSOLT 28.85 dB

【関数定義】

レベル iLv の学習済み回転角を単一レベルのひな型ネットワークにコピー

```
function protoLv = cplevelangles(fullnet,protoTemplate,iLv)
% レベル毎に独立な回転角(Angles)を持つ多レベル網から、レベル iLv 分の値だけを
% 単一レベル(nLevels=1)のひな型ネットワークにコピーする。
protoLv = protoTemplate;
prefixFrom = sprintf('Lv%d_',iLv);
for i = 1:height(fullnet.Learnables)
    name = fullnet.Learnables.Layer(i);
    if startsWith(name,prefixFrom)
        newname = "Lv1_" + extractAfter(name,prefixFrom);
        j = find(protoLv.Learnables.Layer==newname & ...
            protoLv.Learnables.Parameter==fullnet.Learnables.Parameter(i));
        if ~isempty(j)
            protoLv.Learnables.Value(j) = fullnet.Learnables.Value(i);
        end
    end
end
end
end
```

単一レベル分の要素画像(atomic images)の抽出

```
function [atomicImages, mRows, mCols] = getatomicimages(synthesisnet,
patchsize, scale)
% GETATOMICIMAGES 学習済み NSOLT 合成網（単一レベル）から要素画像を計算する
% 各要素インパルス入力に対する合成網の応答を求めることで、単一レベル分の
% ラティス構造と等価な多チャネル FIR フィルタ（畳み込みカーネル）を構築する。
import saivdr.dcnns.*
if nargin < 3 || isempty(scale)
    scale = 1;
end
expfinallayer = '^Lv1_Cmp1+_V0~?$';
expidctlayer = '^Lv\d+_E0~?$';
nLayers = height(synthesisnet.Layers);
nLevels = 0;
nComponents = 1;
for iLayer = 1:nLayers
    layer = synthesisnet.Layers(iLayer);
    if ~isempty(regexpi(layer.Name,expfinallayer,'once'))
        nChannels = layer.NumberOfChannels;
        decFactor = layer.DecimationFactor;
    end
    if ~isempty(regexpi(layer.Name,expidctlayer,'once'))
        nLevels = nLevels + 1;
        if nLevels == 1
            nComponents = layer.NumInputs;
        end
    end
end
end
end
```

```

nChsPerLv = sum(nChannels);
nChsTotal = nLevels*(nChsPerLv-1)+1;
DIMENSION = 2;
MARGIN = 2;
if nargin < 2 || isempty(patchsize)
    estPpOrder = floor([1 1]*sqrt(nLayers/(DIMENSION*nLevels)));
    estKernelExt = decFactor.*(estPpOrder+1);
    for iLv = 2:nLevels
        estKernelExt = (estKernelExt-1).*(decFactor+1)+1;
    end
    maxDecFactor = decFactor.^nLevels;
    patchsize = (ceil(estKernelExt./maxDecFactor)+MARGIN).*maxDecFactor;
end
atomicImages = zeros([patchsize 1 nChsTotal], 'single');
dls = cell(nLevels+1,1);
for iRevLv = nLevels:-1:1
    if iRevLv == nLevels
        dls{nLevels+1} = dldarray(zeros([patchsize./(decFactor.^nLevels)
nComponents], 'single'), 'SSC');
        dls{nLevels} = dldarray(zeros([patchsize./(decFactor.^nLevels)
nComponents*(nChsPerLv-1)], 'single'), 'SSC');
    else
        dls{iRevLv} = dldarray(zeros([patchsize./(decFactor.^iRevLv)
nComponents*(nChsPerLv-1)], 'single'), 'SSC');
    end
end
idx = 1;
dld = dls;
dld{nLevels+1}(round(end/2), round(end/2), 1:nComponents) =
ones(1,1,nComponents);
atomicImages(:, :, 1:nComponents, idx) =
extractdata(synthesisnet.predict(dld{:}));
idx = idx+1;
for iRevLv = nLevels:-1:1
    for iAtom = 1:nChsPerLv-1
        dld = dls;
        for iCmp = 1:nComponents
            dld{iRevLv}(round(end/2), round(end/2),
(iCmp-1)*(nChsPerLv-1)+iAtom) = 1;
        end
        atomicImages(:, :, 1:nComponents, idx) =
extractdata(synthesisnet.predict(dld{:}));
        idx = idx+1;
    end
end
atomicImages = scale * atomicImages;
mRows = 2^(nextpow2(sqrt(nChsTotal))-1);
mCols = ceil(nChsTotal/mRows);
end

```

単一レベル分の合成畳み込み（転置畳み込み＋周期折返し）

```
function y = synthesisnsolt_conv(x, W_syn, decFactor, padSz, szSub)
x = cast(x, 'like', W_syn);
x_3d = reshape(x, szSub);
x_dl = dlarray(x_3d, 'SSC');
bias_s = zeros(1, 'like', W_syn);
y_full_dl = dltranspconv(x_dl, W_syn, bias_s, 'Stride', decFactor,
    'Cropping', 0);
y_full = extractdata(y_full_dl); % [H_full W_full 1]
% 周期折返し（NSOLTの周期境界条件を再現）
p_H = padSz(1); p_W = padSz(2);
N_H = (szSub(1)-1)*decFactor(1) + size(W_syn,1) - 2*p_H;
N_W = (szSub(2)-1)*decFactor(2) + size(W_syn,2) - 2*p_W;
y = y_full(p_H+1:p_H+N_H, p_W+1:p_W+N_W, 1);
y(end-p_H+1:end,:) = y(end-p_H+1:end,:) + y_full(1:p_H, p_W+1:p_W+N_W, 1);
y(1:p_H,:) = y(1:p_H,:) + y_full(p_H+N_H+1:end,
    p_W+1:p_W+N_W, 1);
y(:,end-p_W+1:end) = y(:,end-p_W+1:end) + y_full(p_H+1:p_H+N_H, 1:p_W, 1);
y(:,1:p_W) = y(:,1:p_W) + y_full(p_H+1:p_H+N_H,
    p_W+N_W+1:end, 1);
y(end-p_H+1:end,end-p_W+1:end) = y(end-p_H+1:end,end-p_W+1:end) +
    y_full(1:p_H, 1:p_W, 1);
y(end-p_H+1:end,1:p_W) = y(end-p_H+1:end,1:p_W) +
    y_full(1:p_H, p_W+N_W+1:end, 1);
y(1:p_H,end-p_W+1:end) = y(1:p_H,end-p_W+1:end) +
    y_full(p_H+N_H+1:end, 1:p_W, 1);
y(1:p_H,1:p_W) = y(1:p_H,1:p_W) +
    y_full(p_H+N_H+1:end, p_W+N_W+1:end, 1);
end
```

単一レベル分の分析畳み込み（周期拡張＋畳み込み）

```
function x = analysisnsolt_conv(y, W_syn, decFactor, padSz)
y = cast(y, 'like', W_syn);
if ismatrix(y); y = reshape(y, [size(y,1) size(y,2) 1]); end
p_H = padSz(1); p_W = padSz(2);
y_pad = padarray(y, [p_H p_W 0], 'circular', 'both'); % 周期拡張
y_dl = dlarray(y_pad, 'SSC');
bias_a = zeros(size(W_syn,4),1, 'like', W_syn);
x_dl = dlconv(y_dl, W_syn, bias_a, 'Stride', decFactor, 'Padding', 0);
x = extractdata(x_dl);
x = x(:);
end
```

NSOLT 合成処理関数（レベルごとの畳み込みを再帰的に適用する階層的な方式）

```
function y = synthesisnsolt(x, config)
nLevels = config.nLevels;
nChsPerLv = config.nChsPerLv;
decFactor = config.decFactor;
```

```

padSz = config.padSz;
szOrg = config.szOrg;
nAc = nChsPerLv - 1;
sidx = 1;
acParts = cell(1,nLevels);
for iLv = 1:nLevels
    szSub = szOrg./(decFactor.^iLv);
    n = prod(szSub)*nAc;
    acParts{iLv} = reshape(x(sidx:sidx+n-1),[szSub nAc]);
    sidx = sidx+n;
end
szDc = szOrg./(decFactor.^nLevels);
dc = reshape(x(sidx:sidx+prod(szDc)-1),szDc);
for iLv = nLevels:-1:1
    szSub = szOrg./(decFactor.^iLv);
    combined = cat(3,dc,acParts{iLv}); % [DC, AC_1..AC_nAc] の順で要素画像の並び
    と一致させる
    dc = synthesisnsolt_conv(combined(:),config.W{iLv},decFactor,padSz,
[szSub nChsPerLv]);
end
y = dc;
end

```

NSOLT 分析処理関数（レベルごとの畳み込みを再帰的に適用する階層的方式）

```

function x = analysisnsolt(y,config)
nLevels = config.nLevels;
nChsPerLv = config.nChsPerLv;
decFactor = config.decFactor;
padSz = config.padSz;
dc = y;
acParts = cell(1,nLevels);
for iLv = 1:nLevels
    combined = analysisnsolt_conv(dc,config.W{iLv},decFactor,padSz);
    szSub = size(dc)./decFactor;
    combined = reshape(combined,[szSub nChsPerLv]);
    dc = combined(:, :, 1);
    acParts{iLv} = combined(:, :, 2:end);
end
parts = cell(1,nLevels+1);
for iLv = 1:nLevels
    parts{iLv} = acParts{iLv}(:);
end
parts{nLevels+1} = dc(:);
x = cat(1,parts{:});
end

```

ハード閾値処理

```

function y = hardthresh(x,K)

```

```

v = sort(abs(x(:)), 'descend');
thk = v(K);
y = (abs(x)>thk).*x;
end

```

深層学習配列に対する繰返しハード閾値処理(IHT)のバッチ処理

```

function newdata = iht4patchds(oldtbl, analyzer, synthesizer, sparsityRatio)
% IHT for InputImage in randomPatchExtractionDatastore
%
nInputs = length(synthesizer.InputNames);

% Apply IHT process for every input patch
restbl = removevars(oldtbl, 'InputImage');
dlv = dlarray(cat(4, oldtbl.InputImage{:}), 'SSCB');
[~, dlcoefs{1:nInputs}] = iht4dlarray(dlv, analyzer, synthesizer, sparsityRatio);
coefs = cellfun(@(x) permute(num2cell(extractdata(x), 1:3), [4 1 2
3]), dlcoefs, 'UniformOutput', false);
%
n_imgs = length(oldtbl.InputImage);
coefarray = cell(n_imgs, nInputs);
for i_img = 1:n_imgs
    for i_input = 1:nInputs
        coefarray{i_img, i_input} = coefs{i_input}{i_img};
    end
end
% Output as a cell in order to make multiple-input datastore
newdata = [ coefarray table2cell(restbl) ];
end

```

深層学習配列に対する繰返しハード閾値処理(IHT)

```

function [dly, varargout] =
iht4dlarray(dlX, analyzer, synthesizer, sparsityRatio)
% IHT Iterative hard thresholding
%
nInputs = length(synthesizer.InputNames);
szBatch = size(dlX, 4);

% Iterative hard thresholding w/o normalization
% (A Parseval tight frame is assumed)
gamma = (1.-1e-3);
nIters = 10;
nCoefs = floor(sparsityRatio*numel(dlX(:, :, :, 1)));
[dlcoefs{1:nInputs}] =
analyzer.predict(dlarray(zeros(size(dlX), 'like', dlX), 'SSCB'));
% IHT
for iter=1:nIters
    % Gradient descent
    dly = synthesizer.predict(dlcoefs{1:nInputs});
    [grad{1:nInputs}] = analyzer.predict(dlX-dly);
end

```

```

    dlcoefs = cellfun(@(x,y) x+gamma*y,dlcoefs,grad,'UniformOutput',false);
    % Hard thresholding
    coefvecs = cellfun(@(x) extractdata(reshape(x,
[],szBatch)),dlcoefs,'UniformOutput',false);
    srtdabscoefs = sort(abs(cell2mat(coefvecs.')),1,'descend');
    thk = reshape(srtdabscoefs(nCoefs,:),1,1,1,szBatch);
    dlcoefs = cellfun(@(x) (abs(x)>thk).*x,dlcoefs,'UniformOutput',false);
    % Monitoring
    %checkSparsity =...
    %nnz(srtdabscoefs>srtdabscoefs(nCoefs,:))/numel(dlX)<=sparsityRatio;
    %assert(checkSparsity)
    %fprintf("IHT(%d) MSE: %6.4f\n",iter,mse(dlX,dly));
end
varargout = dlcoefs;
end

```

NSOLT ネットワークの随伴関係の確認

```

function checkadjointrelation(analysislgraph,synthesislgraph,nLevels,szInput)
import saivdr.dcn.*
x = rand(szInput,'single');
% Assemble analyzer
analysislgraph4predict = analysislgraph;
for iLayer = 1:length(analysislgraph4predict.Layers)
    layer = analysislgraph4predict.Layers(iLayer);
    if contains(layer.Name,"Lv"+nLevels+"_DcOut") || ...
        ~isempty(regexp(layer.Name,'^Lv\d+_AcOut','once'))
        analysislgraph4predict =
analysislgraph4predict.replaceLayer(layer.Name,...
        regressionLayer('Name',layer.Name));
    end
end
analysisnet4predict = assembleNetwork(analysislgraph4predict);

% Assemble synthesizer
synthesislgraph4predict = synthesislgraph;
synthesisnet4predict = assembleNetwork(synthesislgraph4predict);

% Analysis and synthesis process
[s{1:nLevels+1}] = analysisnet4predict.predict(x);
if isvector(s{end-1})
    s{end-1} = permute(s{end-1},[1,3,2]);
end
y = synthesisnet4predict.predict(s{:});

% Evaluation
display("MSE: " + num2str(mse(x,y)))
end

```

直交マッチング追跡関数の定義

```

function x = omp(y,Phi,nCoefs)
% Initialization
nAtoms = size(Phi,2);
normsq = sum(Phi.^2,1).'; % ||dm||^2, m=1,...,nAtoms (constant over the loop)
x = zeros(nAtoms,1);
r = y;
supp = false(nAtoms,1);
k = 0;
while k < nCoefs
    % Matching process (vectorized over all atoms at once)
    g = Phi.'*r; % ym=<dm,r>
    a = g./normsq; % Normalize  $\alpha_m = y_m / ||dm||^2$ 
    e = (r.'*r) - g.*a; % <r-dm/||dm||^2,r>
    e(supp) = Inf; % Exclude atoms already in the support
    % Minimum value search (pursuit)
    [~,mmin] = min(e);
    % Update the support
    supp(mmin) = true;
    subPhi = Phi(:,supp);
    x(supp) = subPhi \ y; % Least squares via QR, faster than pinv
    % Residual
    r = y - Phi*x;
    % Update
    k = k + 1;
end
end

```

NSOLT ネットワークからのツリーレベル情報の抽出

```

function nLevels = extractnumlevels(nsoltnet)
import saivdr.dcn.*

% Extraction of information
expidctlayer = '^Lv\d+_E0~?$';
nLevels = 0;
nLayers = height(nsoltnet.Layers);
for iLayer = 1:nLayers
    layer = nsoltnet.Layers(iLayer);
    if ~isempty(regexpi(layer.Name,expidctlayer,'once'))
        nLevels = nLevels + 1;
    end
end
end
end

```

NSOLT ネットワークからのストライド情報の抽出

```

function decFactor = extractdecfactor(nsoltnet)
import saivdr.dcn.*

% Extraction of information
expfinalayer = '^Lv1_Cmp1+_V0~?$';

```

```

nLayers = height(nsoltnet.Layers);
for iLayer = 1:nLayers
    layer = nsoltnet.Layers(iLayer);
    if ~isempty(regexp(layer.Name,expfinallayer,'once'))
        decFactor = layer.DecimationFactor;
    end
end
end

```

NSOLT ネットワークからのチャンネル数情報の抽出

```

function nChannels = extractnumchannels(nsoltnet)
import saivdr.dcn.*

% Extraction of information
expfinallayer = '^Lv1_Cmp1+_V0~?$';
nLayers = height(nsoltnet.Layers);
for iLayer = 1:nLayers
    layer = nsoltnet.Layers(iLayer);
    if ~isempty(regexp(layer.Name,expfinallayer,'once'))
        nChannels = layer.NumberOfChannels;
    end
end
end

```

© Copyright, 2023-2026, Shogo MURAMATSU, All rights reserved.